

CYBERSECURITY INTERNSHIP | WEEK 2 TASK

Log Analysis Practice on Sample Logs

Prepared by	Mahnoor Mehtab Abbasi- Group Leader
Role in task	Analyst and Group Leader
Analysis dates	02 - 03 August 2026
Platform	Kali Linux - Apache 2.4.68, PHP 8.4, MariaDB 11.8
Dataset A	Elastic public sample Apache access log - 10,000 entries, 2.3 MB
Dataset B	Locally generated Apache log from a DVWA instance attacked by the analyst
Toolset	wget, grep, awk, sort, uniq, wc, curl, bash scripting, Python 3
Deliverables	Findings report (this document) and proposed detection rules
Classification	Training exercise - public sample data and a self-owned isolated lab only

Contents

1. Executive Summary
2. Objective, Scope and Requirement Mapping
3. Environment and Tooling

PART A - PUBLIC APACHE SAMPLE LOG

4. Preparing the Apache Access Log
5. Understanding the Log Format
6. Part A Findings (F1 - F7)
7. Part A Automation - detect_404.sh

PART B - SELF-GENERATED DVWA LOG

8. Why a Second, Self-Generated Dataset
9. Building the DVWA Lab
10. Simulating the Attacks
11. Part B Findings (F8 - F15)
12. Building a Python Detection Engine
13. Automated Response - prevention_engine.py

SYNTHESIS

14. Consolidated Findings Summary
15. Recommended Detection Rules (R1 - R6)
16. Limitations and Analyst Notes
17. Conclusion and Next Steps
- A. Appendix A - Full Command Reference
- B. Appendix B - Python Source Code
- C. Appendix C - Screenshot Index

1. Executive Summary

This exercise analyzed web-server logs to identify suspicious activity and to convert those observations into deploy-able detection rules. It was carried out in two complementary parts.

Part A examined a 10,000-entry public Apache access log. Because the traffic in it is genuine internet background noise, it shows what real, unsolicited hostile activity looks like - but the analyst has no way to know for certain what any given request was truly attempting.

Part B addressed that gap by building a deliberately vulnerable web application (DVWA) on an isolated Kali Linux machine, attacking it deliberately, and then analyzing the log the attacks produced. Here the ground truth is known exactly, so each detection could be validated rather than merely proposed.

Headline findings

- **Part A revealed automated reconnaissance, not compromise.** 213 of 10,000 requests (2.13%) returned 404, clustered on WordPress, and Drupal administrative paths that do not exist on the server, plus probes for a legacy file-manager component with known upload vulnerabilities. Every probe failed.
- **The noisiest source was benign.** The single largest contributor of failures - 60 of 213 - was a Chef Client automation agent repeatedly requesting one removed file. Volume alone is a poor discriminator; counting distinct paths separates broken automation from genuine scanning cleanly.
- **Part A predicted a blind spot; Part B proved it.** No HTTP 401 response existed anywhere in Data set A, meaning a status-code filter cannot see application-level login attacks. In Part B, a real brute-force attack against DVWA produced exactly zero 401s - every login attempt returned 200 or 302.
- **Successful exploitation looks identical to failure in the status code.** The successful DVWA login returned HTTP 200, the same as every failed one. It was distinguishable only by a change in response size (2,145 bytes versus 2,103) and by a follow-on request for the account avatar image that only loads once authenticated.
- **Injection payloads survive in the log, but only URL-encoded.** SQL injection and cross-site scripting attempts appear in the query string as percent-encoded text. A plain-text search for the payload finds nothing; decoding first finds all of it.
- **Command injection left no trace at all.** The payload travelled in a POST body, which Apache does not record. This is a genuine and unavoidable limitation of access-log-only visibility.

Key numbers at a glance

Measurement	Value	What it tells us
Dataset A - total records	10,000	Baseline for every ratio quoted in Part A
Dataset A - HTTP 401 responses	0	No HTTP-auth brute force, and a real telemetry blind spot
Dataset A - HTTP 404 responses	213	2.13% of traffic - elevated and worth investigating
Dataset A - sources over threshold	2	208.91.156.11 (60, benign) and 144.76.95.39 (14, suspicious)
Dataset B - total records	97	Small, but every entry has known ground truth
Dataset B - HTTP 401 responses	0	Confirms the Part A prediction under a real brute-force attack

Dataset B - brute-force hits	18	Requests to the DVWA brute-force endpoint from 127.0.0.1
Dataset B - directory probes	14	Sensitive-path requests from ::1 using curl
Dataset B - tool user agents	16	Requests presenting curl or a spoofed sqlmap identity

Overall assessment

Dataset A: low-intensity opportunistic scanning of the kind that reaches every internet-facing host continuously. No compromise indicator was found - every probe returned 404. The value of the finding is as early warning: hosts that scan today frequently exploit tomorrow.

Dataset B: all five simulated attack classes were detected, but only after the detection logic was adapted to what the log actually records. Three of the six recommended rules in Section 15 exist specifically because a naive status-code approach would have missed the attack entirely.

2. Objective, Scope and Requirement Mapping

The assigned task defined four steps and two deliverables. The table below maps each requirement to the section that satisfies it, so a reviewer can verify completeness at a glance.

Task requirement	Where it is addressed
1. Obtain or generate a sample log file (Apache / Nginx access logs)	Section 4 (obtained) and Sections 9-10 (generated)
2. Identify suspicious patterns - repeated failed logins	Findings F1 and F8-F9 - covered in both datasets
2. Identify suspicious patterns - scanning behaviour	Findings F3, F4, F7 and F13
2. Identify suspicious patterns - unusual user agents	Findings F6 and F14
3. Summarise findings with supporting log excerpts	Sections 6, 11 and 14 - excerpts plus terminal screenshots throughout
4. Recommend 2-3 detection rules based on findings	Section 15 - six rules with thresholds, tuning notes and working code
Tools: Python / grep / Excel for log parsing	grep and awk in Sections 6 and 11; Python engine in Sections 12-13
Deliverable: findings report	This document
Deliverable: sample detection-rule ideas	Section 15, with runnable implementations in Sections 7, 12 and 13

Scope statement

- In scope: one publicly published Apache access log in Combined Log Format, analysed offline.
- In scope: a DVWA instance installed on the analyst's own isolated Kali Linux machine, attacked only over the loopback interface (127.0.0.1 and ::1).
- Out of scope: any live, third-party or production system. No scanning, probing or testing was performed against any host not owned by the analyst.
- Out of scope: attribution. IP addresses are reported exactly as observed, with no claim about who operates them.

Legal and ethical note

DVWA is published expressly so that security practitioners can attack it. It was run on an isolated local machine and reached only through the loopback address, so no third-party system was ever touched. Its own documentation warns against exposing it to any network, and that warning was followed.

3. Environment and Tooling

All work was carried out from a terminal on Kali Linux. No commercial SIEM was used. The point of the exercise is that a competent analyst can reach real conclusions from raw logs using tools that ship with every Linux system, and can then automate those conclusions in a few dozen lines of Python.

Tool	Role in this analysis
wget	Retrieved the Dataset A sample log over HTTPS
wc -l	Counted total records, and counted matches when piped from grep
head / tail	Previewed records to confirm the log format before parsing
grep / grep -E	Filtered records by status code, path and payload pattern
awk	Field-aware extraction - client IP, request line, status code, user agent
sort uniq -c	Grouped and counted repeated values - the core of frequency analysis
bash + nano	Turned proven one-liners into repeatable detection scripts
curl	Generated controlled scanning traffic against the local DVWA instance
Apache2 / PHP / MariaDB	Served DVWA and, critically, wrote the Dataset B access log
Python 3 (urllib, collections)	Decoded percent-encoded payloads and implemented the detection engine

Why command-line tools before Python, and Python before a SIEM

A dashboard hides its reasoning. Building each query by hand forces you to know exactly which field is being tested and why a threshold sits where it does. Every Python rule in Section 12 began as a shell one-liner that was first proven against real data, and every threshold in Section 15 can be traced back to a specific observation in Section 6 or 11.

PART A - PUBLIC APACHE SAMPLE LOG

Part A analyses a real Apache access log published by Elastic. The traffic is genuine, unsolicited internet activity, which makes it an honest picture of what a public web server receives every day.

4. Preparing the Apache Access Log

4.1 Creating an isolated working directory

Analysis begins in a dedicated directory. Keeping evidence separate from the rest of the filesystem prevents accidental modification of the source data and makes the working set easy to archive later.

```
cd Downloads
mkdir log-analysis
cd log-analysis
```

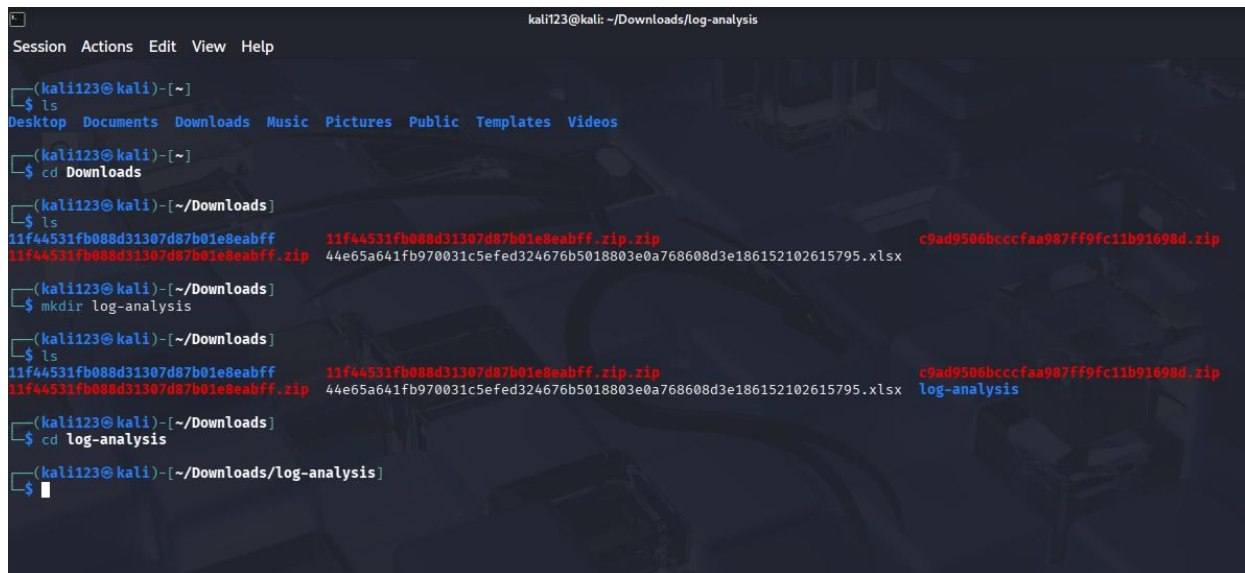


Figure 1 - Creating and entering the dedicated log-analysis working directory.

4.2 Obtaining the sample log

The sample is the Apache access log published in Elastic's public examples repository. It is a genuine web-server log, which is what makes it useful: the suspicious traffic in it is real background noise rather than something inserted artificially for a lab.

```
wget
https://raw.githubusercontent.com/elastic/examples/master/Common%20Data%20Formats/apache_logs/ap
ache_logs
```

The transfer completed with HTTP 200 and 2,370,789 bytes were written to disk.

```

(kali123@kali)-[~/Downloads]
$ cd log-analysis

(kali123@kali)-[~/Downloads/log-analysis]
$ wget https://raw.githubusercontent.com/elastic/examples/master/Common%20Data%20Formats/apache_logs/apache_logs
--2026-08-02 04:20:08-- https://raw.githubusercontent.com/elastic/examples/master/Common%20Data%20Formats/apache_logs/apache_logs
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.108.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2370789 (2.3M) [text/plain]
Saving to: 'apache_logs'

apache_logs 100%[=====] 2.26M 5.08MB/s in 0.4s

2026-08-02 04:20:10 (5.08 MB/s) - 'apache_logs' saved [2370789/2370789]

(kali123@kali)-[~/Downloads/log-analysis]
$ ls
apache_logs

```

Figure 2 - Downloading the sample log.

4.3 Validating the file before analysing it

Never start parsing a file you have not verified. Three quick checks confirm the download is intact, is the format expected, and is large enough to be meaningful.

Check 1 - size on disk

```
ls -lh
```

```

(kali123@kali)-[~/Downloads/log-analysis]
$ ls
apache_logs

(kali123@kali)-[~/Downloads/log-analysis]
$ ls -lh
total 2.3M
-rw-rw-r-- 1 kali123 kali123 2.3M Aug  2 04:20 apache_logs

```

Figure 3 - The file is 2.3 MB, matching the byte count reported by wget.

Check 2 - format inspection

Reading the first lines confirms the file uses Apache Combined Log Format. This step is what makes the later awk commands reliable: you cannot safely reference field 9 until you have seen that field 9 really is the status code.

```
head apache_logs
```



```
(kali123@kali) - [~/Downloads/log-analysis]
$ head apache_logs
83.149.9.216 - - [17/May/2015:10:05:03 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1" 200 203023 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:43 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-dashboard3.png HTTP/1.1" 200 171717 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:47 +0000] "GET /presentations/logstash-monitorama-2013/plugin/highlight/highlight.js HTTP/1.1" 200 26185 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:12 +0000] "GET /presentations/logstash-monitorama-2013/plugin/zoom-js/zoom.js HTTP/1.1" 200 7697 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:07 +0000] "GET /presentations/logstash-monitorama-2013/plugin/notes/notes.js HTTP/1.1" 200 2892 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:34 +0000] "GET /presentations/logstash-monitorama-2013/images/sad-medic.png HTTP/1.1" 200 430406 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:57 +0000] "GET /presentations/logstash-monitorama-2013/css/fonts/Roboto-Bold.ttf HTTP/1.1" 200 38720 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:50 +0000] "GET /presentations/logstash-monitorama-2013/css/fonts/Roboto-Regular.ttf HTTP/1.1" 200 41820 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:24 +0000] "GET /presentations/logstash-monitorama-2013/images/frontend-response-codes.png HTTP/1.1" 200 52878 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
83.149.9.216 - - [17/May/2015:10:05:50 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-dashboard.png HTTP/1.1" 200 321631 "http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
```

Figure 4 - First records of the log: IP, timestamp, request, status, size, referrer and user agent.

Check 3 - record count

The record count is the denominator for every percentage quoted later. Without it, a raw number such as 213 has no meaning.

```
wc -l apache_logs
```

```
(kali123@kali) - [~/Downloads/log-analysis]
$ wc -l apache_logs
10000 apache_logs

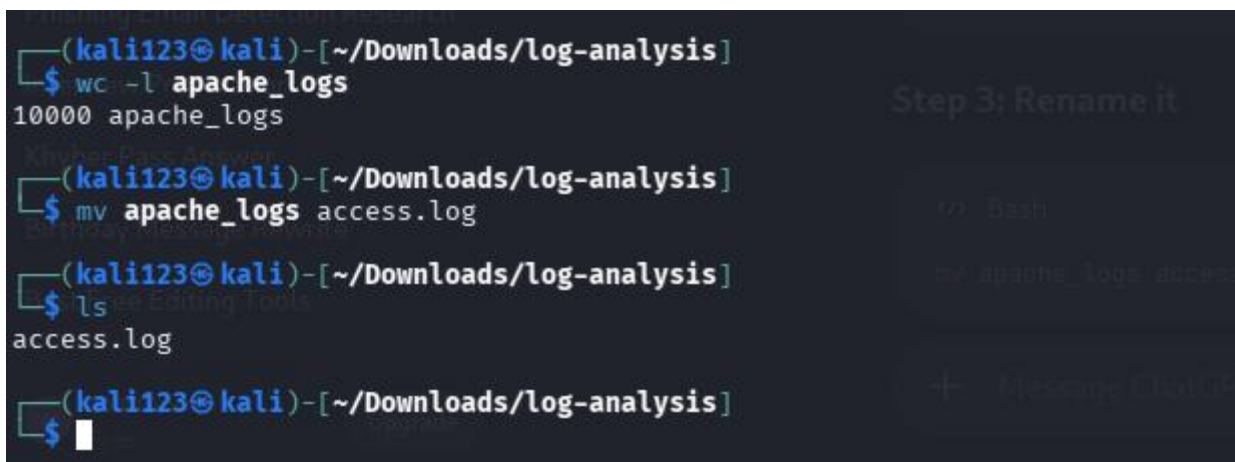
(kali123@kali) - [~/Downloads/log-analysis]
$
```

Figure 5 - Exactly 10,000 records. This is the baseline for all ratio calculations.

4.4 Normalising the filename

The file was renamed to access.log so the commands in this report match what a reader would encounter on a real Apache server, where the default path is /var/log/apache2/access.log.

```
mv apache_logs access.log
ls
```

A terminal window with a dark background and light blue text. The prompt is '(kali123@kali) - [~/Downloads/log-analysis]'. The first command is '\$ wc -l apache_logs', which outputs '10000 apache_logs'. The second command is '\$ mv apache_logs access.log'. The third command is '\$ ls', which outputs 'access.log'. The prompt is now '\$ ' with a cursor.

```
(kali123@kali) - [~/Downloads/log-analysis]
$ wc -l apache_logs
10000 apache_logs

(kali123@kali) - [~/Downloads/log-analysis]
$ mv apache_logs access.log

(kali123@kali) - [~/Downloads/log-analysis]
$ ls
access.log

(kali123@kali) - [~/Downloads/log-analysis]
$
```

Figure 6 - Renaming the dataset to mirror a production Apache path.

5. Understanding the Log Format

Every command that follows depends on knowing which whitespace-separated field holds which value. Apache Combined Log Format is fixed and predictable, which is exactly why awk suits it so well. Below is one real record from the dataset, broken into its parts.

A single record, as written by Apache

```
83.149.9.216 - - [17/May/2015:10:05:03 +0000] "GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1" 200 203023
"http://semicomplete.com/presentations/logstash-monitorama-2013/"
"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 ..."
```

Field-by-field breakdown

awk field	Value in the example	Meaning and analytical use
\$1	83.149.9.216	Client IP. Groups activity by source - the basis of Rule R1.
\$2, \$3	- -	Identd and HTTP auth username. A real username here means an authenticated request.
\$4, \$5	[17/May/2015:10:05:03 +0000]	Timestamp and timezone. Turns a count into a rate.
\$6	"GET	HTTP method. A burst of HEAD requests is a scanner fingerprint (F4).
\$7	/presentations/.../kibana-search.png	Requested path, including the query string. The single most valuable field.
\$8	HTTP/1.1"	Protocol version. HTTP/1.0 from a supposedly modern browser is an anomaly.
\$9	200	Status code. 401 auth failed, 403 forbidden, 404 not found. The core filter.
\$10	203023	Response size in bytes. Decisive in Finding F9 - it revealed a successful login.
quoted 2	"http://semicomplete.com/..."	Referrer. Absent on nearly all automated requests - but see F11 for a trap.
quoted 3	"Mozilla/5.0 (Macintosh; ...)"	User agent. Trivially forged, but forged badly by most tooling (F6, F14).

Two different extraction styles, and when to use each

Splitting on whitespace (awk's default) gives \$1 and \$9 cleanly. But the request line and the user agent are wrapped in double quotes and contain spaces, so whitespace splitting tears them apart. For those, the field separator is switched to the double-quote character, which makes the request line field 2 and the user agent field 6. Both techniques appear throughout this report.

What Apache does NOT log

The access log records the request line only. POST bodies, request headers other than Referer and User-Agent, cookies and response bodies are all absent. This single fact explains Finding F12, where a successful command-injection attack left no usable trace whatsoever.

6. Part A Findings

Findings are numbered F1 to F7. Each states what was looked for, the command used, the evidence returned, and - most importantly - what the result actually means. A negative result is reported with the same weight as a positive one, because knowing what is absent is part of the picture.

F1 - No authentication failures present (negative finding)

Hypothesis tested: the task specifically asks about repeated failed logins, so the first search was for HTTP 401 Unauthorized, which Apache returns when HTTP Basic authentication fails.

```
grep ' 401 ' access.log
grep ' 401 ' access.log | wc -l
```



```
(kali123@kali)~[/Downloads/log-analysis]
$ ls
access.log

(kali123@kali)~[/Downloads/log-analysis]
$ grep ' 401 ' access.log

(kali123@kali)~[/Downloads/log-analysis]
$ grep ' 401 ' access.log | wc -l
0

(kali123@kali)~[/Downloads/log-analysis]
$
```

Figure 7 - The 401 search returns no output at all, and the count confirms zero matches.

Result: zero matches.

Interpretation: this does not mean nobody tried to log in. It means the server does not use HTTP Basic authentication, so nothing here would ever emit a 401. Application-level logins - WordPress, DVWA, virtually every modern web application - are handled inside the application. A failed WordPress login returns HTTP 200 with an error rendered in the page body, invisible to any status-code filter.

Analytical consequence: credential attacks against this class of application must be detected by a different signal: request volume to the login endpoint, POST-to-GET ratio, or application logs. This prediction is tested directly in Part B, Finding F8 - and confirmed.

Analyst note - the value of a negative result

It is tempting to discard a search that returns nothing. Do not. Recording that 401s were checked and found absent is what lets the reader trust that the 404 conclusion was reached by elimination rather than by guessing, and it documents a genuine, actionable gap in the telemetry.

F2 - Elevated volume of 404 Not Found responses

Hypothesis tested: with authentication failures ruled out, the next indicator is a cluster of requests for resources that do not exist. Scanners work from wordlists, and most wordlist entries are absent from any given server, so scanning produces 404s in bulk.

```
grep ' 404 ' access.log
```

```
(kali123@kali) [~/Downloads/log-analysis]
$ grep ' 404 ' access.log
36.249.73.185 - - [17/May/2015:10:05:22 +0000] "GET /doc/index.html?org=elasticsearch/action/search/SearchResponse.html HTTP/1.1" 404 294 "-" Mozilla/5.0
(compatible; Googlebot/2.1; +http://www.google.com/bot.html)
208.91.156.11 - - [17/May/2015:11:05:05 +0000] "GET /files/logstash/logstash-1.3.2-monolithic.jar HTTP/1.1" 404 324 "-" Chef Client/10.18.2 (ruby-1.9.3-p3
27; ohai-6.16.0; x86_64-linux; +http://opscode.com)"
111.199.235.239 - - [17/May/2015:13:05:25 +0000] "GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.1" 404 3
54 "http://semicomplete.com/presentations/logstash-puppetconf-2012/" Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_5) AppleWebKit/536.30.1 (KHTML, like Gec
b) Version/6.0.5 Safari/536.30.1"
111.199.235.239 - - [17/May/2015:13:05:32 +0000] "GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.1" 404 3
54 "http://semicomplete.com/presentations/logstash-puppetconf-2012/" Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_5) AppleWebKit/536.30.1 (KHTML, like Gec
b) Version/6.0.5 Safari/536.30.1"
208.91.156.11 - - [17/May/2015:13:05:04 +0000] "GET /files/logstash/logstash-1.3.2-monolithic.jar HTTP/1.1" 404 324 "-" Chef Client/10.18.2 (ruby-1.9.3-p3
27; ohai-6.16.0; x86_64-linux; +http://opscode.com)"
144.76.194.187 - - [17/May/2015:13:05:28 +0000] "GET /wp-login.php HTTP/1.0" 404 292 "-" "-"
144.76.194.187 - - [17/May/2015:13:05:37 +0000] "GET /administrator/index.php HTTP/1.0" 404 303 "-" "-"
208.91.156.11 - - [17/May/2015:15:05:00 +0000] "GET /files/logstash/logstash-1.3.2-monolithic.jar HTTP/1.1" 404 324 "-" Chef Client/10.18.2 (ruby-1.9.3-p3
27; ohai-6.16.0; x86_64-linux; +http://opscode.com)"
208.91.156.11 - - [17/May/2015:16:05:23 +0000] "GET /files/logstash/logstash-1.3.2-monolithic.jar HTTP/1.1" 404 324 "-" Chef Client/10.18.2 (ruby-1.9.3-p3
27; ohai-6.16.0; x86_64-linux; +http://opscode.com)"
74.208.180.23 - - [17/May/2015:16:05:50 +0000] "GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.1" 404 364
"Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; SV1; .NET CLR 2.0.50727; InfoPath.1)"
36.249.73.135 - - [17/May/2015:17:05:19 +0000] "GET /projects/xdotool%3E HTTP/1.1" 404 7861 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google
.com/bot.html)"
122.166.142.108 - - [17/May/2015:17:05:50 +0000] "GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.1" 404 3
54 "http://semicomplete.com/presentations/logstash-puppetconf-2012/" Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_5) AppleWebKit/537.73.11 (KHTML, like Gec
ko) Version/6.1.1 Safari/537.73.11"
195.250.34.144 - - [17/May/2015:17:05:24 +0000] "GET /wp-login.php HTTP/1.1" 404 292 "-" Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.1 (KHTML, like Gecko
) Chrome/24.0.1290.1 Safari/537.13"
195.250.34.144 - - [17/May/2015:17:05:38 +0000] "GET /administrator/ HTTP/1.1" 404 294 "-" Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.1 (KHTML, like Gec
ko) Chrome/24.0.1290.1 Safari/537.13"
195.250.34.144 - - [17/May/2015:17:05:50 +0000] "GET /admin.php HTTP/1.1" 404 289 "-" Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.1 (KHTML, like Gecko) C
hrome/24.0.1290.1 Safari/537.13"
122.166.142.108 - - [17/May/2015:17:05:09 +0000] "GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.1" 404 3
54 "http://semicomplete.com/presentations/logstash-puppetconf-2012/" Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_5) AppleWebKit/537.73.11 (KHTML, like Gec
ko) Version/6.1.1 Safari/537.73.11"
208.91.156.11 - - [17/May/2015:17:05:32 +0000] "GET /files/logstash/logstash-1.3.2-monolithic.jar HTTP/1.1" 404 324 "-" Chef Client/10.18.2 (ruby-1.9.3-p3
27; ohai-6.16.0; x86_64-linux; +http://opscode.com)"
195.211.162.22 - - [17/May/2015:18:05:31 +0000] "GET /projects/xdotool/+++++++Result:+chosen+nickname+%22awarovadoms%22;sent; HTTP/1.0" 404 3
75 "http://www.semicomplete.com/projects/xdotool/+++++++Result:+chosen+nickname+%22awarovadoms%22;sent;" Mozilla/4.0 (compatible; MSIE 5.5;
Windows NT 5.0)"
```

Figure 8 - Raw 404 records. The pattern is visible even at a glance: wp-login.php, /administrator/index.php and /admin.php requested from several sources.

```
grep ' 404 ' access.log | wc -l
```

```
(kali123@kali) [~/Downloads/log-analysis]
$ grep ' 404 ' access.log | wc -l
213
```

Figure 9 - 213 responses with status 404.

Result: 213 of 10,000 requests returned 404 - 2.13% of all traffic.

Interpretation: a healthy public website typically runs well under 1% 404s, mostly stale bookmarks, missing favicons and dead inbound links. At 2.13%, with the failures clustered on paths no legitimate visitor would ever type, this is elevated. The raw number is not the finding - the composition of those 404s, examined next, is the finding.

Watch the spacing in the grep pattern

The pattern is ' 404 ' with a leading and trailing space, not 404. Without the spaces, grep would also match a response size of 404 bytes, a timestamp containing 404, or a path such as /error404.html, inflating the count with false positives. Precision in the filter is what makes the resulting statistic defensible.

F3 - WordPress and admin-panel enumeration (scanning behaviour)

Hypothesis tested: if the 404s come from scanning, the requested paths will repeat across sources and will target well-known administrative endpoints. Counting the request line by frequency answers this directly.

```
grep ' 404 ' access.log | awk -F'"' '{print $2}' | sort | uniq -c | sort -nr
```

The pipeline reads: filter to 404s; split each line on the double-quote character and take field 2 (the full request line); sort so identical requests become adjacent; count each unique request; then sort numerically in reverse so the most frequent appear first.

```
(kali123@kali) - [~/Downloads/log-analysis]
$ grep ' 404 ' access.log | awk -F'"' '{print $2}' | sort | uniq -c | sort -nr
61 GET /files/logstash/logstash-1.3.2-monolithic.jar HTTP/1.1
31 GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.1
6 GET /wp/wp-admin/ HTTP/1.1
6 GET /wp-login.php?action=register HTTP/1.0
6 GET /wp-admin/ HTTP/1.1
6 GET /blog/wp-admin/ HTTP/1.1
5 GET /wordpress/wp-admin/ HTTP/1.1
4 GET /wp-login.php HTTP/1.1
4 GET /image/logstash.png HTTP/1.1
4 GET /admin.php HTTP/1.1
4 GET /administrator/ HTTP/1.1
3 POST /blog/geekery/pybloxom-mdade-vim-hack.html/trackingback/ HTTP/1.1
3 GET /browserconfig.xml HTTP/1.1
3 GET /apple-touch-icon.png HTTP/1.1
2 GET /wp-login.php HTTP/1.0
2 GET /scripts/%22file://$file/%22 HTTP/1.0
2 GET /scripts/%22$%7BWEBLOC%7D/view.php?file=$%7Blinkto%7D%5C%22 HTTP/1.0
2 GET /scripts/%22$%7BWEBLOC%7D/view.php?file=$%7Blinkto%7D%5C%22 HTTP/1.0
2 GET /projects/xdotool/xdotool HTTP/1.1
2 GET /presentations/logstash-puppetconf-2013/css/font/fontawesome-webfont.woff?v=3.2.1 HTTP/1.1
2 GET /presentations/logstash-puppetconf-2013/css/font/fontawesome-webfont.ttf?v=3.2.1 HTTP/1.1
2 GET /presentations/logstash-puppetconf-2013/css/font/fontawesome-webfont.svg HTTP/1.1
2 GET /misc/nmh/%22file://$file/%22 HTTP/1.0
2 GET /apple-touch-icon-precomposed.png HTTP/1.1
```

Figure 10 - Ranked list of failed requests. The administrative cluster sits immediately below the two high-count legitimate misses.

```
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/fckeditor/editor/fckeditor.html HTTP/1.1
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/admin/FCKeditor/_samples/default.html HTTP/1.1
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/admin/FCKeditor/editor/fckeditor.html HTTP/1.1
1 GET /user/register HTTP/1.0
1 GET /sitemap.xml HTTP/1.1
1 GET /projects/xdotool/+++++Result:chosen+nickname+%22awarovadoms%22;sent; HTTP/1.0
1 GET /projects/xdotool%3E HTTP/1.1
1 GET /projects/xd HTTP/1.1
1 GET /projects/securitrack/config.xsl HTTP/1.1
1 GET /projects/securitrack/config.xml HTTP/1.1
1 GET /presentations/vim/%094 HTTP/1.0
1 GET /presentations/logstash-puppetconf-2012/lib/font/league_gothic-webfont.ttf%20format(%22truetype%22 HTTP/1.1
1 GET /presentations/logstash-puppetconf-2012/images/office-space-printer-beat-down-gif.gif HTTP/1.0
1 GET /node/add/blog HTTP/1.1
1 GET /geekery/find-that-lost-screen-session.html HTTP/1.1
1 GET /geekery/find-that-lost-screen-session.html HTTP/1.0
1 GET /geekery/find-that-lost-screen-session-2.html HTTP/1.0
1 GET /files/xdotool/xdotool-%25 HTTP/1.0
1 GET /files/logstash/logstash-%25 HTTP/1.0
1 GET /doc/org/elasticsearch/action/support/master/TransportMasterNodeOperationAction.html HTTP/1.1
1 GET /doc/index.html?org/elasticsearch/action/search/SearchResponse.html HTTP/1.1
1 GET /blog/tags/+ HTTP/1.1
1 GET /blog/tags/g HTTP/1.0
1 GET /blog/tags/2010 HTTP/1.1
1 GET /blog/geekery/www.csh.rit.edu/~robertp HTTP/1.1
1 GET /blog/geekery/jquery-i**terface-puffer.html HTTP/1.1
1 GET /blog/geekery/jquery-interface-/ppuffer.html HTTP/1.1
1 GET /blog/geekery/jquery-interface-/p%20ppuffer.html HTTP/1.1
1 GET /blog/geekery/jquery-interface- HTTP/1.0
1 GET /blog/geekery/jquery-interface-%20 HTTP/1.1
1 GET /blog/geekery/jquery-i HTTP/1.1
1 GET /blog/geekery/htorg/style/iphone.css?p://www.semicomplete.com/about/ HTTP/1.1
1 GET /blog/geekery/%E2%80%A6 HTTP/1.0
1 GET /blog/geekery/2!/? HTTP/1.1
1 GET /articles/ssh-????????????????/ HTTP/1.0
1 GET /articles/dy HTTP/1.1
1 GET /apple-touch-icon-120x120-precomposed.png HTTP/1.1
1 GET /apple-touch-icon-120x120.png HTTP/1.1
1 GET /about/wal:RecentChanges HTTP/1.1
1 GET /about/tal:RecentChanges HTTP/1.1
```

Figure 11 - Continuation of the ranked list, showing the long tail of single-occurrence probe paths.

The administrative cluster

Extracted from the ranked output, the following paths were requested and all returned 404:

```

6 GET /wp-login.php?action=register HTTP/1.0
6 GET /wp-admin/ HTTP/1.0
6 GET /wp-admin/ HTTP/1.1
5 GET /blog/wp-admin/ HTTP/1.1
5 GET /wordpress/wp-admin/ HTTP/1.1
4 GET /wp-login.php HTTP/1.1
4 GET /admin.php HTTP/1.1
4 GET /administrator/ HTTP/1.1
2 GET /wp-login.php HTTP/1.0
2 GET /administrator/index.php HTTP/1.0
1 GET /user/register HTTP/1.0

```

Interpretation - the most important finding in Part A. Three characteristics mark this as automated scanning rather than human traffic:

- **Prefix rotation.** The same target, wp-admin, was requested at /wp-admin/, /blog/wp-admin/ and /wordpress/wp-admin/. A human wanting the admin panel tries once. A scanner iterates a list of common installation sub-directories because it does not know where WordPress lives - or whether it is installed at all.
- **Cross-platform coverage.** /wp-login.php is WordPress, /administrator/ is Joomla, /admin.php is generic, /user/register is Drupal. No visitor looks for four CMS platforms on one site. A scanner tests all of them because it costs nothing to try.
- **Protocol inconsistency.** The same path appears over both HTTP/1.0 and HTTP/1.1. Real browsers have not issued HTTP/1.0 in decades. Scripted clients and older scanning tools still do, and the mixture indicates more than one tool or operator.

Corroborating evidence from the raw records: source 195.250.34.144 requested /wp-login.php, /administrator/ and /admin.php within roughly twenty-six seconds. Three different CMS admin endpoints from one address inside half a minute is not browsing - that is a scanner working through a list.

```

195.250.34.144 - - [17/May/2015:17:05:24 +0000] "GET /wp-login.php HTTP/1.1" 404 292
195.250.34.144 - - [17/May/2015:17:05:38 +0000] "GET /administrator/ HTTP/1.1" 404 294
195.250.34.144 - - [17/May/2015:17:05:50 +0000] "GET /admin.php HTTP/1.1" 404 289

```

Why it matters: the registration probe, /wp-login.php?action=register, goes a step beyond discovery. It tests whether open user registration is enabled - a common initial foothold, because a self-registered low-privilege account is often enough to reach an authenticated vulnerability. The intent behind these requests is not curiosity.

F4 - Legacy file-manager probing via HEAD requests

A distinct block of single-occurrence entries targets FCKeditor, a legacy rich-text editor whose bundled file manager is associated with well-documented arbitrary-file-upload vulnerabilities.

```
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/fckeditor/editor/fckeditor.html
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/fckeditor/_samples/default.html
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/admin/FCKeditor/editor/fckeditor.html
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/admin/FCKeditor/_samples/default.html
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/include/fckeditor/editor/
  filemanager/connectors/test.html
1 HEAD /blog/geekery/ec2-reserved-vs-ondemand.html/js/fckeditor/editor/fckeditor.html
```

Interpretation: two signals overlap here.

- **The method is HEAD, not GET.** HEAD asks for headers only, with no response body. A scanner uses it deliberately: it confirms whether a file exists using a fraction of the bandwidth and produces smaller, less conspicuous log entries. Browsers effectively never issue HEAD for a page, so a run of them is a strong tool fingerprint on its own.
- **The path pattern is combinatorial.** The same base article URL is combined with `/fckeditor/`, `/js/fckeditor/`, `/include/fckeditor/` and `/admin/FCKeditor/` - including a case variation. That is a nested loop over a wordlist, not a person clicking links.

Why it matters: the specific file requested, `filemanager/connectors/test.html`, is the FCKeditor connector test page. If it responds, an attacker can often upload a web shell through it. This is not general reconnaissance - it is a check for one specific exploitable component. All returned 404, so the component is not present.

F5 - File-inclusion and path-traversal probe strings

Several requests in the long tail contain encoded characters and protocol handlers that have no legitimate place in a URL:

```
2 GET /scripts/%22file:/$file%22 HTTP/1.0
2 GET /scripts/%22$%7BWEBLOC%7D/view.php?file=$%7Blinkto%7D%5C%22 HTTP/1.0
2 GET /misc/nmh/%22file:/$file%22 HTTP/1.0
1 GET /articles/ssh-????????????????????/ HTTP/1.0
1 GET /projects/xdotool%3E HTTP/1.1
```

Interpretation: `%22` is an encoded double quote and `%7B` / `%7D` are encoded braces. The strings `file://` and `$file` are classic building blocks of a local-file-inclusion test: if the application interpolates the parameter into a file path, the probe reveals it. The `view.php?file=` pattern is a textbook LFI target. The trailing `%3E` is an encoded greater-than sign, a probe for cross-site-scripting filtering.

Why it matters: these are not typos. Encoded quote and brace characters do not appear in ordinary navigation. Whether they came from an automated scanner or from a template-injection bug on a linking site, any request containing `file://` or an encoded quote in the path deserves an alert - it is cheap to detect and almost never benign. Part B, Finding F10, shows the same encoding technique carrying a live SQL injection payload.

F6 - Unusual and implausible user agents

The task specifically calls out unusual user agents. Three distinct categories appear in the 404 traffic.

Category 1 - obsolete browser string on 2015 traffic

```
195.211.161.22 - - [17/May/2015:18:05:31 +0000] "GET /projects/xdotool/++++++
+++++++Result:+chosen+nickname+%22awarovadoms%22;sent; HTTP/1.0" 404 175
"Mozilla/4.0 (compatible; MSIE 5.5; Windows NT 5.0)"
```

Why this is suspicious: Windows NT 5.0 is Windows 2000 and MSIE 5.5 shipped in 2000; Microsoft support ended in 2010. A genuine client running it in 2015 is implausible - a spam or scanning tool with a hard-coded default string is the ordinary explanation. The request itself confirms it: the path contains a long run of plus characters and the text 'chosen nickname ... sent', the residue of an automated form-submission tool writing its status message into a URL. Note also the HTTP/1.0 protocol, inconsistent with the browser it claims to be.

Category 2 - automation client, high volume

```
208.91.156.11 - - [17/May/2015:11:05:05 +0000] "GET /files/logstash/logstash-
1.3.2-monolithic.jar HTTP/1.1" 404 324 "-" "Chef Client/10.18.2 (ruby-1.9.3-p327;
ohai-6.16.0; x86_64-linux; +http://opscode.com)"
```

Why this matters, and why it is probably NOT an attack: Chef Client is a configuration-management agent. This source produced 60 of the 213 404s - the single largest contributor - but every one of them requests the same missing file, a Logstash JAR removed from the server. This is a fleet of managed servers repeatedly failing to fetch a dependency, not an adversary. Reporting it as an attack would be a false positive, and a costly one.

The most important analyst lesson in this report

The loudest source in the data is the benign one. If a rule counting total 404s per IP were deployed untuned, 208.91.156.11 would alert every single day and the rule would be muted within a week - taking the genuine detections with it. This is exactly why Rule R1 in Section 15 counts DISTINCT paths instead: broken automation hits one path many times, a scanner hits many paths a few times each. One line of logic separates the two behaviours cleanly.

Category 3 - declared crawler (requires verification)

```
66.249.73.185 - - [17/May/2015:10:05:22 +0000] "GET /doc/index.html?org/
elasticsearch/action/search/SearchResponse.html HTTP/1.1" 404 294 "-"
"Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
```

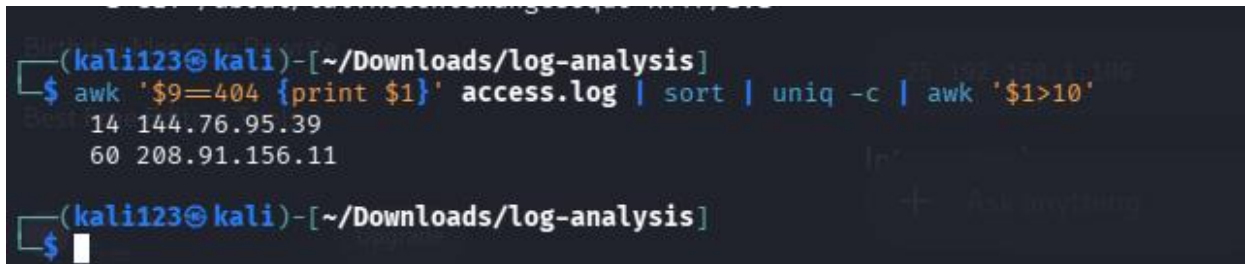
Assessment: the 66.249.x.x range does belong to Google and the requested paths are stale documentation links, which is normal crawler behaviour. This is assessed as legitimate. It is included to make a procedural point: the user-agent string is attacker-controlled and proves nothing. Any crawler claim must be validated by reverse DNS on the source IP followed by a forward lookup of the result. Attackers impersonate Googlebot precisely because analysts wave it through.

F7 - Concentration of failures by source address

Hypothesis tested: are the 213 failures spread thinly across the internet, or concentrated in a few sources? Concentration is what makes a source-based rule viable.

```
awk '$9==404 {print $1}' access.log | sort | uniq -c | awk '$1>10'
```

The pipeline reads: for every line whose ninth field is exactly 404, print the client IP; sort and count occurrences per IP; keep only groups whose count exceeds ten. That final awk is the detection logic in miniature - it is a threshold.



```
(kali123@kali)-[~/Downloads/log-analysis]
$ awk '$9==404 {print $1}' access.log | sort | uniq -c | awk '$1>10'
14 144.76.95.39
60 208.91.156.11

(kali123@kali)-[~/Downloads/log-analysis]
$
```

Figure 12 - Only two source addresses exceed ten 404 responses.

Source IP	404 count	Assessment
208.91.156.11	60	BENIGN - Chef Client automation repeatedly requesting one removed file. High volume, single path, consistent user agent. Recommend allow-listing and fixing the missing artefact.
144.76.95.39	14	SUSPICIOUS - lower volume but spread across varied paths, consistent with wordlist-driven enumeration. Recommend monitoring and correlation with the admin-path cluster in F3.

Interpretation: with only two sources above the threshold, the remaining 139 failures are spread across many addresses at low volume each - including the coordinated-looking admin probes from 195.250.34.144, which sit below ten and are therefore invisible to a pure volume threshold. This is the central limitation of counting alone, and it directly motivates Rule R2, which triggers on path composition rather than volume.

7. Part A Automation - detect_404.sh

A finding that requires an analyst to retype a pipeline is a finding that will be checked once and forgotten. The threshold logic from F7 was therefore saved as an executable script - the first step from ad-hoc analysis toward a repeatable control.

7.1 The script as written

```
#!/bin/bash
echo "=== Suspicious 404 Activity ==="
awk '$9==404 {print $1}' access.log | sort | uniq -c | awk '$1>10'
```

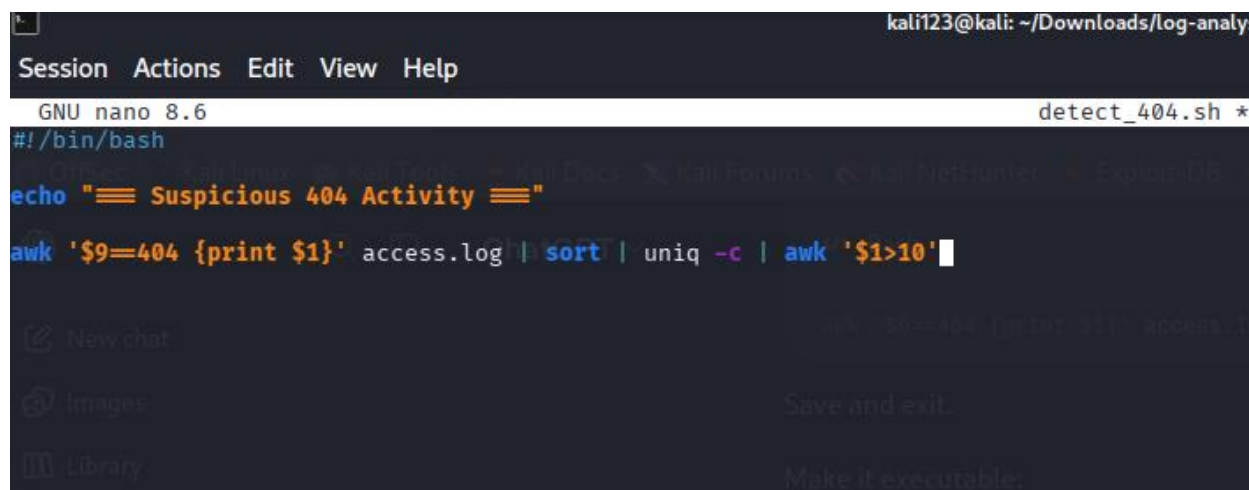


Figure 13 - Writing detect_404.sh in nano. The shebang on line 1 tells the kernel to execute it with bash.

7.2 Making it executable and running it

```
chmod +x detect_404.sh
./detect_404.sh
```

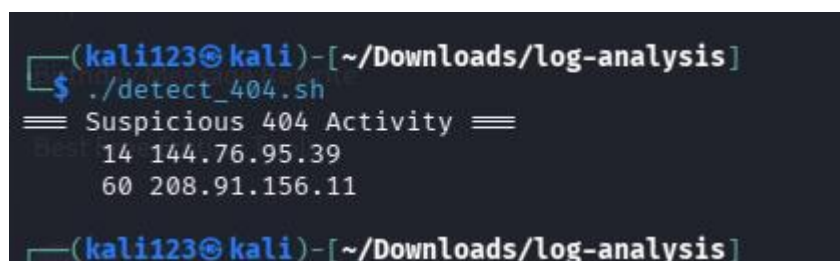


Figure 14 - The script reproduces the F7 result exactly, confirming the automation is faithful to the manual analysis.

The output matches Figure 12 line for line. That equivalence is the point: the script is verified against a known-good manual result before it is trusted.

7.3 Improved version implementing Rule R1

The original inherits the weakness identified in F6 - it counts total requests, so it would flag the benign Chef Client source. The version below implements the distinct-path logic of Rule R1, takes the log file as an argument, and makes the threshold configurable.

```
#!/bin/bash
# detect_scan.sh - flag hosts enumerating many distinct missing paths
# Usage: ./detect_scan.sh [logfile] [threshold]
```

```
LOG="${1:-access.log}"
THRESHOLD="${2:-10}"

if [ ! -r "$LOG" ]; then
    echo "Error: cannot read log file $LOG" >&2
    exit 1
fi

echo "=== Distinct-path 404 enumeration (threshold: $THRESHOLD) ==="

awk '$9==404 {print $1, $7}' "$LOG" \
| sort -u \
| awk '{print $1}' \
| uniq -c \
| awk -v t="$THRESHOLD" '$1 >= t {printf "%-18s %4d distinct missing paths\n", $2, $1}' \
| sort -k2 -nr
```

What changed, and why

sort -u before counting means each IP-and-path pair is counted once, so a source hammering one URL contributes 1 rather than 60. The log file and threshold became arguments so the script can be pointed at any log without editing it. The readability check prevents a silent empty result when the path is wrong - a script that fails quietly is worse than no script at all.

PART B - SELF-GENERATED DVWA LOG

Part B builds a deliberately vulnerable web application on an isolated machine, attacks it under controlled conditions, and analyses the log those attacks produce. Because the analyst performed every attack, the ground truth is known exactly.

8. Why a Second, Self-Generated Dataset

Part A analysed a log that somebody else generated. That limits the exercise in a way worth naming: you can only observe attacks that happened to be present, and you can never confirm what any request was truly attempting. Part B removes both constraints.

- **Known ground truth.** In Part A the nature of every request had to be inferred. In Part B the analyst performs the attack, so what appears in the log can be compared against what was actually done. That is the only way to measure whether a detection rule truly works.
- **Testing the F1 prediction.** Finding F1 predicted that application-level login attacks produce no HTTP 401 and are therefore invisible to a status-code filter. DVWA uses exactly such a login form. Attacking it turns that prediction into a testable claim - and Finding F9 reports the result.
- **Attack classes absent from Dataset A.** Dataset A contained reconnaissance only. Part B adds successful SQL injection, cross-site scripting and command injection, so the report covers exploitation as well as discovery.
- **Rule validation instead of rule proposal.** Running a directory scan against the local instance should trip Rule R1, and admin-path requests should trip Rule R2. If they do not fire, the thresholds are wrong - and they can be corrected before anyone relies on them.

A note on the dataset sizes

Dataset B contains 97 records against Dataset A's 10,000. That is not a flaw. Dataset A answers 'what does hostile traffic look like at scale?'; Dataset B answers 'what does a specific attack look like line by line?'. The two questions need different data, and the second one needs a log small enough to read in full.

9. Building the DVWA Lab

DVWA (Damn Vulnerable Web Application) is a PHP and MySQL application published specifically so that security practitioners can practise against it legally. Building it requires a web server, a PHP runtime and a database - and, importantly for this exercise, installing Apache is what produces the access log to be analysed.

9.1 Refreshing and upgrading the system

The package index is synchronised first so the components installed afterwards come from current repository metadata. Upgrading before installing avoids dependency conflicts between the existing system and the new Apache and PHP packages - a common cause of a broken DVWA setup.

```
sudo apt update
```

```
(kali㉿kali)-[~]
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.3 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [103 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [169 kB]
Fetched 75.0 MB in 25s (2,974 kB/s)
856 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 15 - Package lists refreshed from the kali-rolling repositories; 856 packages reported upgradable.

```
sudo apt upgrade -y
```

```
(kali㉿kali)-[~]
$ sudo apt upgrade -y
The following packages were automatically installed and are no longer required:
  bluez-hcidump          iucode-tool          libntfs-3g89t64      libvidstab1.1      network-manager-gnome
  firmware-marvell-presteria libencode-perl      libopenjph0.27      libx265-215
Use 'sudo apt autoremove' to remove them.

Upgrading:
7zip                      libjansson4          postgresql-18-jit
acl                      libjavascriptcoregtk-4.1-0 postgresql-client-18
adduser                  libjavascriptcoregtk-6.0-1 powermgmt-base
adwaita-icon-theme      libjq1              powershell-empire
alsa-ucm-conf           libjson-c5          procps
amass                   libjsoncpp26        proxytunnel
apparmor                libk5crypto3        publicsuffix
arping                  libkadm5clnt-mit12  pyinstaller
at-spi2-common          libkadm5srv-mit12   pyinstaller-hooks-contrib
at-spi2-core            libkdb5-10t64       python-matplotlib-data
atftpd                  libkrb5-3           python-tables-data
axel                    libkrb5-dev         python-tinycss2-common
base-files              libkrb5support0     python3-aiohappyeyeballs
bind9-dnsutils          liblapack3          python3-aiohttp
bind9-host              liblastlog2-2       python3-aiohttp-socks
bind9-libs              libldap-common      python3-aiomultiprocess
bioutils                libldap2            python3-aioquic
```

Figure 16 - System upgrade in progress.

9.2 Installing and starting the web server

Apache serves DVWA and writes every request it handles to `/var/log/apache2/access.log` in the same Combined Log Format analysed in Part A. The `enable` command registers the service to start at boot by creating a systemd symlink; `start` brings it up immediately.

```
sudo apt install apache2 -y
```

```
(kali㉿kali)-[~]
$ sudo apt install apache2 -y
[sudo] password for kali:
apache2 is already the newest version (2.4.68-1).
apache2 set to manually installed.
The following packages were automatically installed and are no longer required:
  bluez-hcidump          iucode-tool          libopenjph0.27      libx265-215
  firmware-marvell-presteria libntfs-3g89t64      libvidstab1.1      network-manager-gnome
Use 'sudo apt autoremove' to remove them.

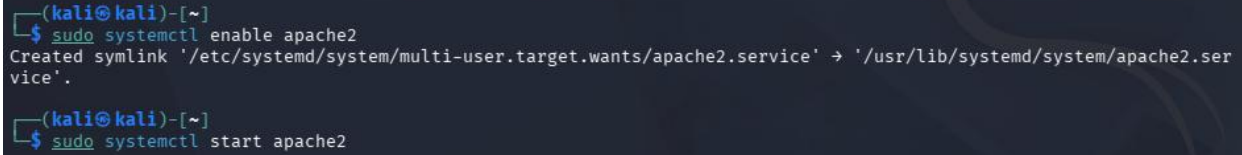
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
```

Figure 17 - Apache 2.4.68 confirmed present.

```
sudo systemctl enable apache2
```

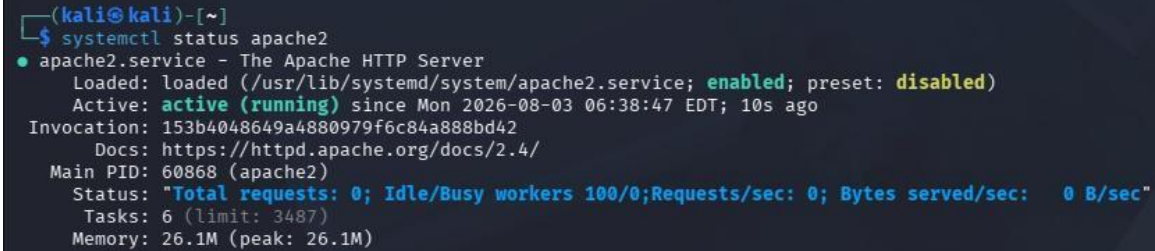


```
sudo systemctl start apache2
systemctl status apache2
```



```
(kali@kali)-[~]
$ sudo systemctl enable apache2
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' → '/usr/lib/systemd/system/apache2.service'.
(kali@kali)-[~]
$ sudo systemctl start apache2
```

Figure 18 - The service is enabled at boot; the confirmed symlink shows it is now persistent.



```
(kali@kali)-[~]
$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-03 06:38:47 EDT; 10s ago
 Invocation: 153b4048649a4880979f6c84a888bd42
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 60868 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
   Tasks: 6 (limit: 3487)
  Memory: 26.1M (peak: 26.1M)
```

Figure 19 - Apache reports active (running). Note 'Total requests: 0' - the log starts clean, which means every entry analysed later was generated by this exercise.

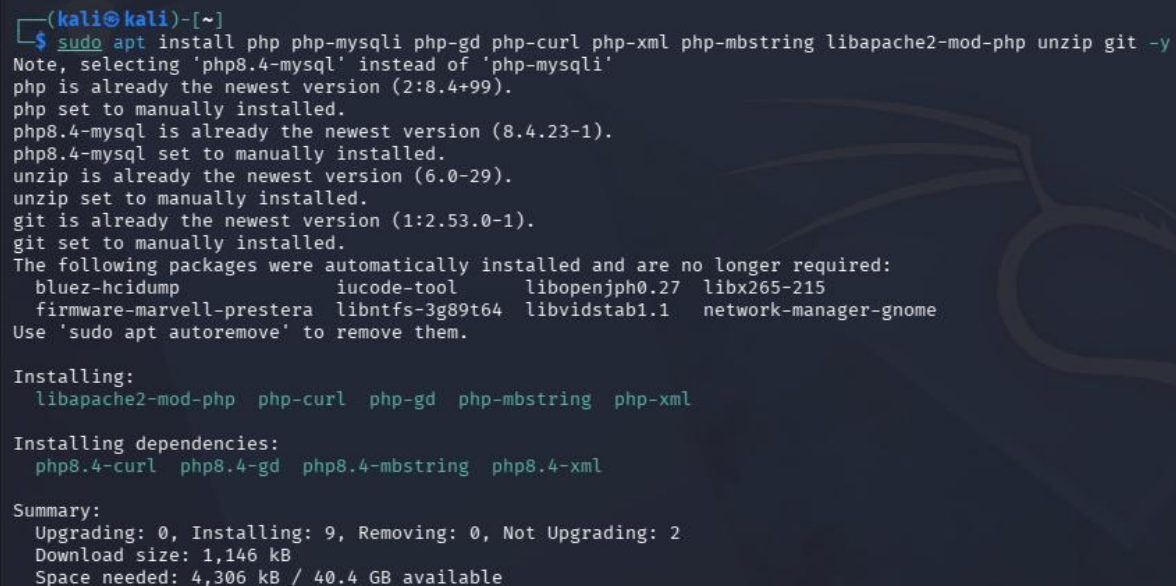
Why the zero-request baseline matters

Starting from a server that has served nothing removes any ambiguity about provenance. In Dataset A, distinguishing attacker traffic from ordinary visitors required inference. Here, there are no ordinary visitors - so every line in the log is either the analyst's browser or the analyst's attack tooling, and any detection that fires can be checked against what was actually done.

9.3 Installing the PHP runtime and supporting packages

DVWA is written in PHP and stores its data in MySQL. The modules below cover database access, image handling, HTTP requests, XML and multibyte strings; git is needed to fetch the application source.

```
sudo apt install php php-mysql php-gd php-curl php-xml php-mbstring \
libapache2-mod-php unzip git -y
```



```
(kali@kali)-[~]
$ sudo apt install php php-mysql php-gd php-curl php-xml php-mbstring libapache2-mod-php unzip git -y
Note, selecting 'php8.4-mysql' instead of 'php-mysql'
php is already the newest version (2:8.4+99).
php set to manually installed.
php8.4-mysql is already the newest version (8.4.23-1).
php8.4-mysql set to manually installed.
unzip is already the newest version (6.0-29).
unzip set to manually installed.
git is already the newest version (1:2.53.0-1).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  bluez-hcidump          iucode-tool          libopenjph0.27      libx265-215
  firmware-marvell-presteria libntfs-3g89t64      libvidstab1.1       network-manager-gnome
Use 'sudo apt autoremove' to remove them.

Installing:
  libapache2-mod-php  php-curl  php-gd  php-mbstring  php-xml

Installing dependencies:
  php8.4-curl  php8.4-gd  php8.4-mbstring  php8.4-xml

Summary:
  Upgrading: 0, Installing: 9, Removing: 0, Not Upgrading: 2
  Download size: 1,146 kB
  Space needed: 4,306 kB / 40.4 GB available
```

Figure 20 - PHP 8.4 and the required extensions installed alongside the Apache PHP module.

9.4 Preparing the database

MariaDB is enabled at boot, then a dedicated database and a dedicated user are created. Granting privileges only on the dvwa database, rather than using the root account, is standard practice and worth doing even in a lab.

```
sudo systemctl enable mariadb
```

```
(kali@kali)-[~]
$ sudo systemctl enable mariadb
Created symlink '/etc/systemd/system/multi-user.target.wants/mariadb.service' → '/usr/lib/systemd/system/mariadb.service'.
```

Figure 21 - MariaDB registered to start at boot.

```
sudo mysql
```

```
CREATE DATABASE dvwa;
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa123';
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

```
(kali@kali)-[~]
$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 11.8.8-MariaDB-1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.003 sec)

MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa123';
Query OK, 0 rows affected (0.007 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> EXIT;
Bye
```

Figure 22 - Database and application user created on MariaDB 11.8. Each statement returns Query OK.

Credentials shown deliberately

The password dvwa123 appears in this report because it belongs to a throwaway local lab that is destroyed after the exercise. It is deliberately weak so that the brute-force simulation in Section 10 succeeds quickly. Nothing resembling this should ever be used outside an isolated practice environment.

9.5 Deploying the application

The default Apache landing page is removed, the DVWA source is cloned into the web root, and ownership is handed to the web-server user so PHP can write its configuration and uploads.

```
cd /var/www/html
sudo rm index.html
```

```
(kali@kali)-[~]
$ cd /var/www/html

(kali@kali)-[/var/www/html]
$ sudo rm index.html
```

Figure 23 - Clearing the default Apache landing page from the web root.

```
sudo git clone https://github.com/digininja/DVWA.git
```

```
(kali@kali)-[/var/www/html]
$ sudo git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA' ...
remote: Enumerating objects: 5746, done.
remote: Total 5746 (delta 0), reused 0 (delta 0), pack-reused 5746 (from 1)
Receiving objects: 100% (5746/5746), 2.76 MiB | 4.04 MiB/s, done.
Resolving deltas: 100% (2857/2857), done.
```

Figure 24 - DVWA source cloned - 5,746 objects received.

```
sudo chown -R www-data:www-data /var/www/html/DVWA
sudo chmod -R 755 /var/www/html/DVWA
```

```
(kali@kali)-[/var/www/html]
$ sudo chown -R www-data:www-data /var/www/html/DVWA

(kali@kali)-[/var/www/html]
$ sudo chmod -R 755 /var/www/html/DVWA
```

Figure 25 - Ownership transferred to the Apache service account and permissions set.

9.6 Configuration and restart

DVWA ships a template configuration file. It is copied into place and edited so the database credentials match those created in Section 9.4; Apache is then restarted to load the changes.

```
cd /var/www/html/DVWA/config
sudo cp config.inc.php.dist config.inc.php
sudo nano config.inc.php
```

```
(kali@kali)-[/var/www/html]
$ cd /var/www/html/DVWA/config

(kali@kali)-[/var/www/html/DVWA/config]
$ sudo cp config.inc.php.dist config.inc.php

(kali@kali)-[/var/www/html/DVWA/config]
$ sudo nano config.inc.php
```

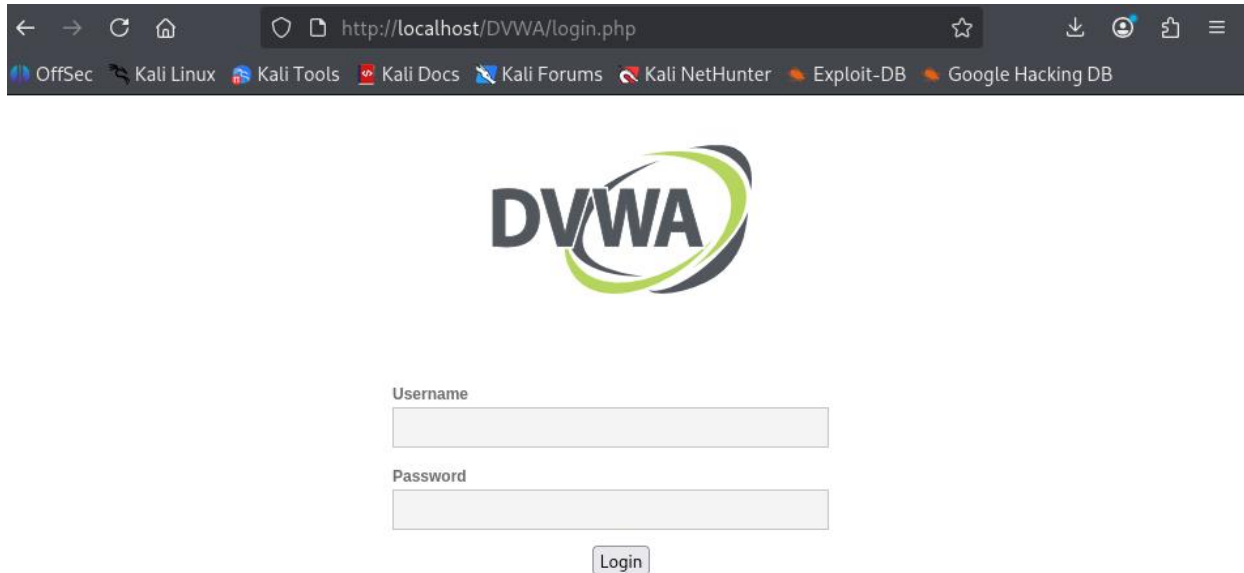
Figure 26 - Creating and editing the DVWA configuration file.

```
sudo systemctl restart apache2
```

```
(kali@kali)-[/var/www/html/DVWA/config]
$ sudo systemctl restart apache2
```

Figure 27 - Restarting Apache to apply the configuration.

9.7 Confirming the application is live



← → ↻ 🏠 🔒 📄 http://localhost/DVWA/login.php ☆ ⬇ 👤 📄 ☰

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

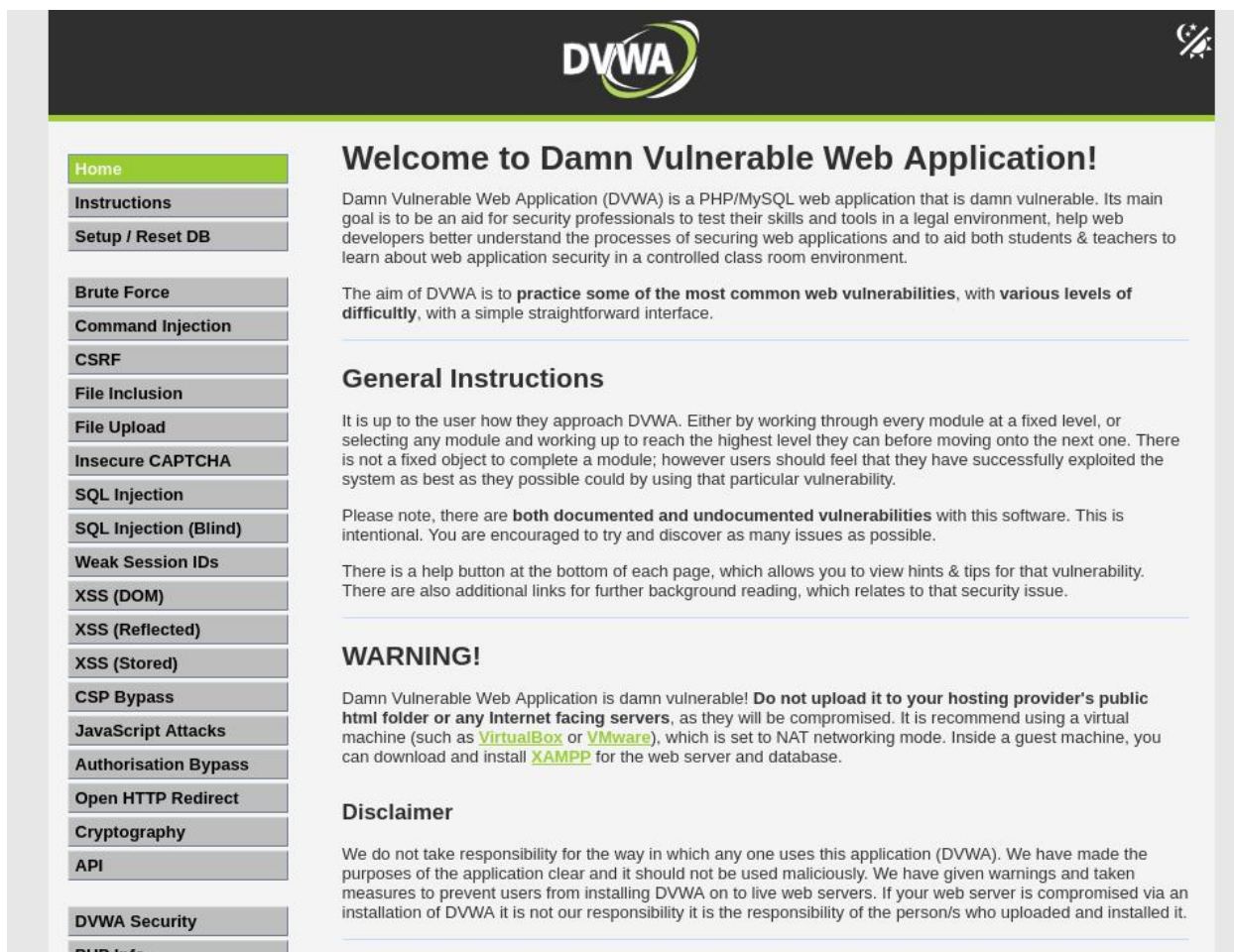
DVWA

Username

Password

Login

Figure 28 - The DVWA login page served at <http://localhost/DVWA/login.php>.



DVWA

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

DVWA Info

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficulty**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

Figure 29 - The DVWA welcome page after authentication, listing the vulnerability modules available for testing.

9.8 Setting the security level

DVWA offers four security levels. Low disables all protections, which is the correct choice here: the objective is to study what a successful attack looks like in the log, not to test whether the application can be defeated.

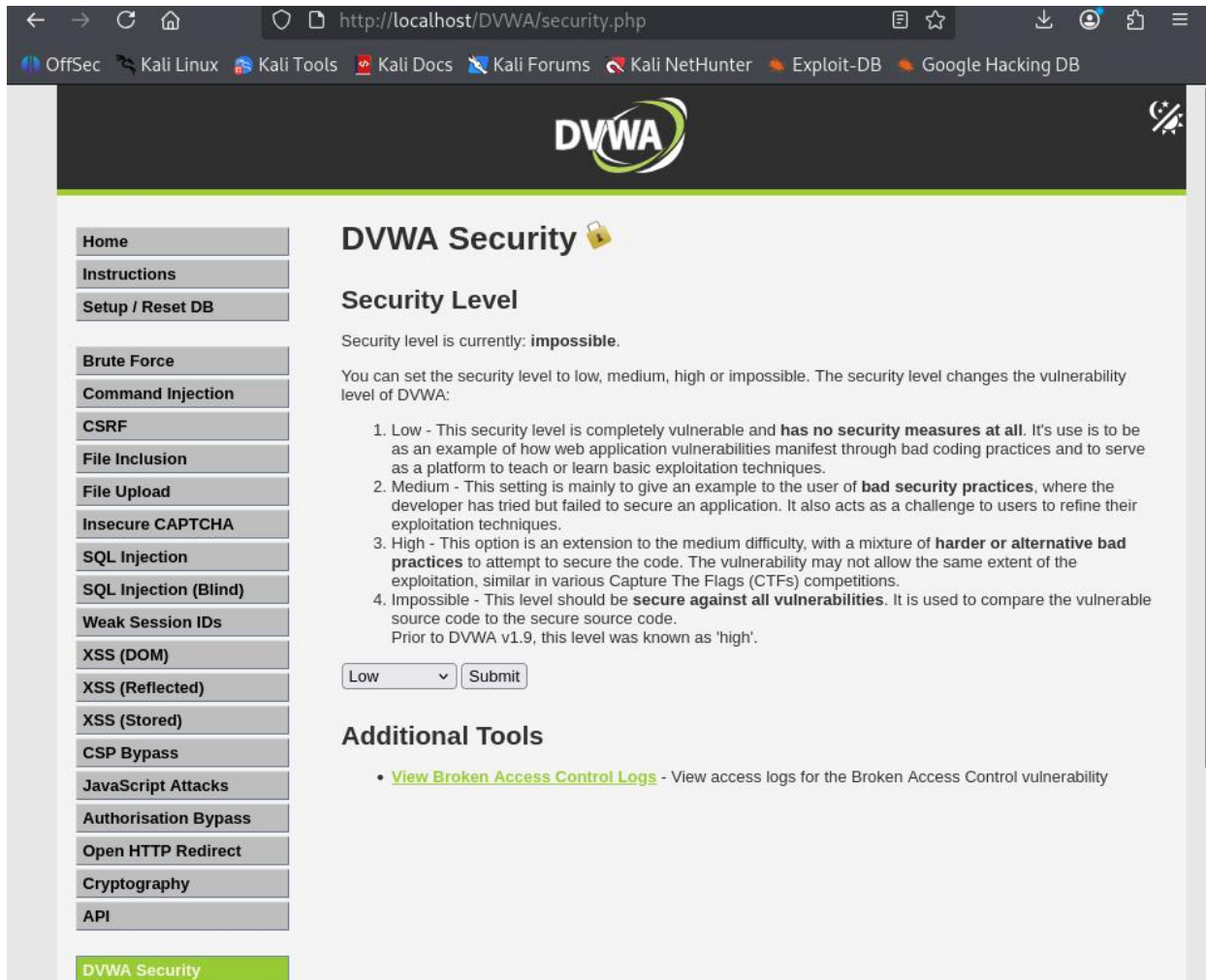


Figure 30 - The DVWA Security page. The level is set to Low so that each attack succeeds and produces a clean, unambiguous log signature.

10. Simulating the Attacks

Five attack classes were executed against the local instance, chosen to cover the three suspicious-pattern categories named in the task brief - repeated failed logins, scanning behaviour and unusual user agents - plus two injection classes that Dataset A did not contain.

Attack executed	Method	Expected log signature
Brute force	Repeated wrong passwords on the DVWA login form	Many POSTs to one endpoint; no 401
SQL injection	Payload 1' OR '1'='1 in the User ID field	Encoded quote and OR in the query string
Reflected XSS	Payload <script>alert(1)</script> in the name field	Encoded angle brackets in the query string
Command injection	Payload 127.0.0.1 && whoami in the ping field	POST body - not recorded by Apache
Directory enumeration	curl requests for /admin, /.git, /phpmyadmin	Repeated 404s, tool user agent, no referrer

10.1 Brute force against the login form

Deliberately wrong credentials were submitted repeatedly to the DVWA Brute Force module. The application responded with a generic failure message and, after several attempts, an account-lockout warning - both rendered inside a normal HTML page.



Figure 31 - Failed login. Note that the application reports the failure in the page body, not in the HTTP status code.

Correct credentials were then supplied, and the application confirmed access to the protected area, reporting the number of attempts made.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Vulnerability: Brute Force

Login

Username:
admin

Password:
••••••••

Login

Welcome to the password protected area **admin**



Warning: Someone might of been brute forcing your account.

Number of login attempts: 3.
Last login attempt was at: 2026-08-03 07:09:38.

More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Figure 32 - Successful login. DVWA itself warns that someone might have been brute forcing the account and records three attempts.

The application knows more than the log does

DVWA counts login attempts, warns about brute forcing, and can lock the account. None of that reasoning reaches the Apache access log - it exists only inside the application. This is the practical argument for collecting application logs alongside web-server logs, and it is why Rule R3 in Section 15 has two independent conditions.

10.2 First look at the generated log

```
cd /var/log/apache2
ls -l
tail -20 access.log
```



```

(kali@kali)-[~]
$ cd /var/log/apache2

(kali@kali)-[/var/log/apache2]
$ ls -l
total 16
-rw-r--r-- 1 root adm 12081 Aug  3 07:10 access.log
-rw-r--r-- 1 root adm 3572 Aug  3 07:04 error.log
-rw-r--r-- 1 root adm  0 Aug  2 20:55 other_vhosts_access.log

(kali@kali)-[/var/log/apache2]
$ tail -20 access.log
127.0.0.1 - - [03/Aug/2026:07:07:42 -0400] "GET /DVWA/security.php HTTP/1.1"
200 2254 "http://localhost/DVWA/index.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:43 -0400] "GET /DVWA/dvwa/images/lock.png HTTP/1.1" 200 1045 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:43 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1707 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:22 -0400] "GET /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 1839 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:31 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2103 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:35 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:20 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2103 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/js/dvwaPage.js HTTP/1.1" 200 900 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/css/main.css HTTP/1.1" 200 1682 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/js/add_event_listeners.js HTTP/1.1" 200 619 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/images/logo.png HTTP/1.1" 200 5330 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/images/theme-light-dark.png HTTP/1.1" 200 1314 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:34 -0400] "POST /DVWA/vulnerabilities/brute/

```

Figure 33 - The Apache access log now contains the traffic generated by the exercise, in the same Combined Log Format used in Part A.

10.3 SQL injection

The classic authentication-bypass payload was submitted to the DVWA SQL Injection module. Because the parameter is interpolated directly into a query, the condition is always true and the application returns every row.

```
User ID: 1' OR '1'='1
```

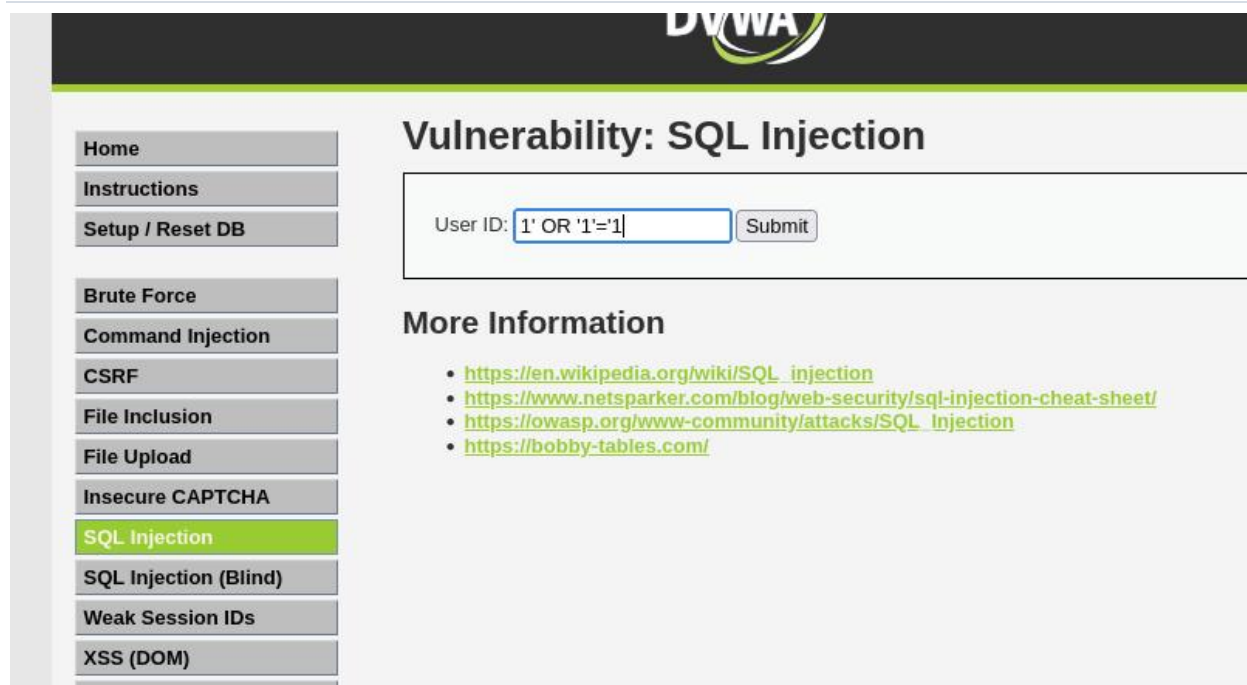


Figure 34 - SQL injection payload submitted through the browser.

10.4 Reflected cross-site scripting

A script tag was submitted to the reflected XSS module, which echoes user input back into the page without encoding it.

```
What's your name? <script>alert(1)</script>
```

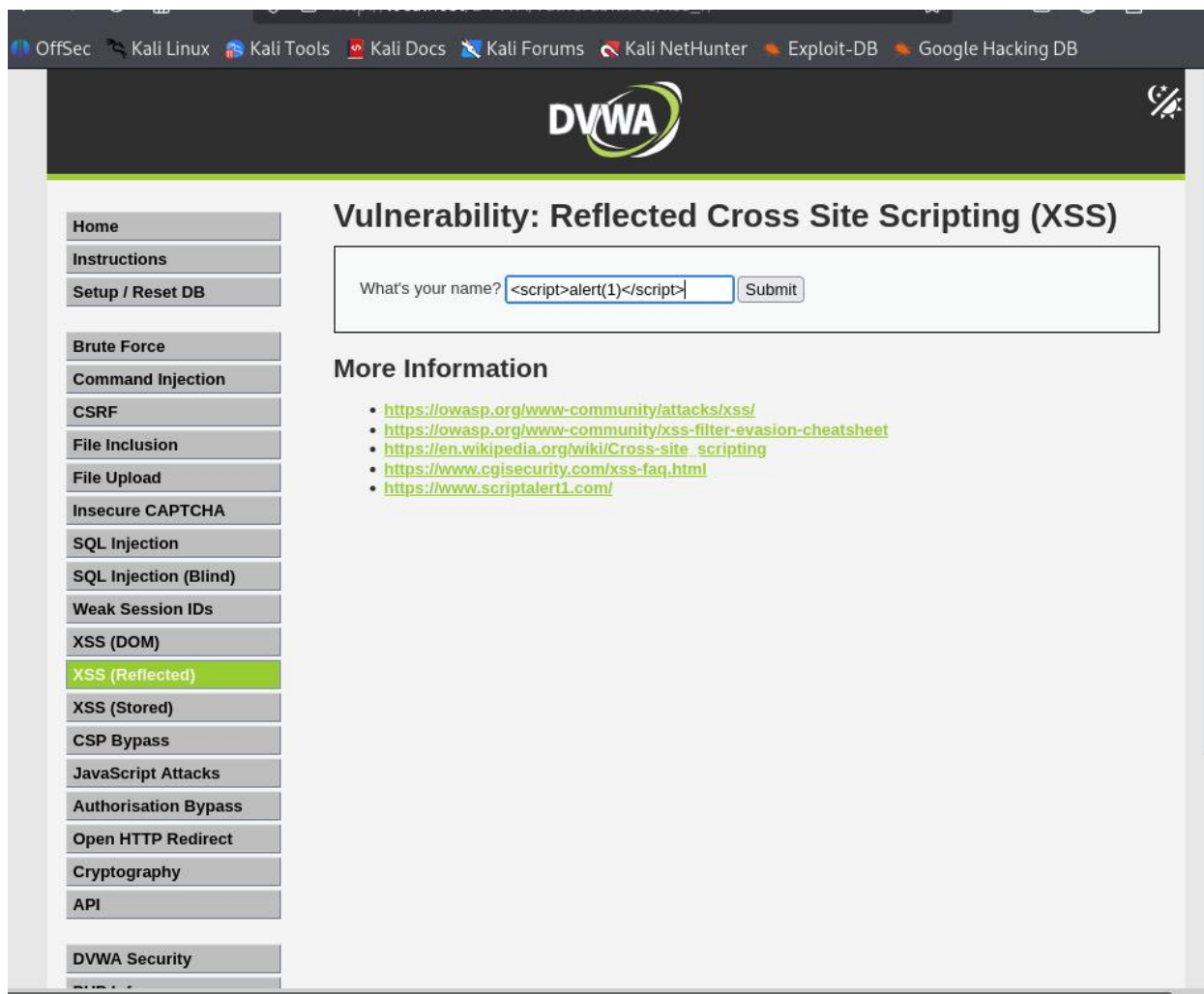


Figure 35 - XSS payload entered into the reflected XSS module.

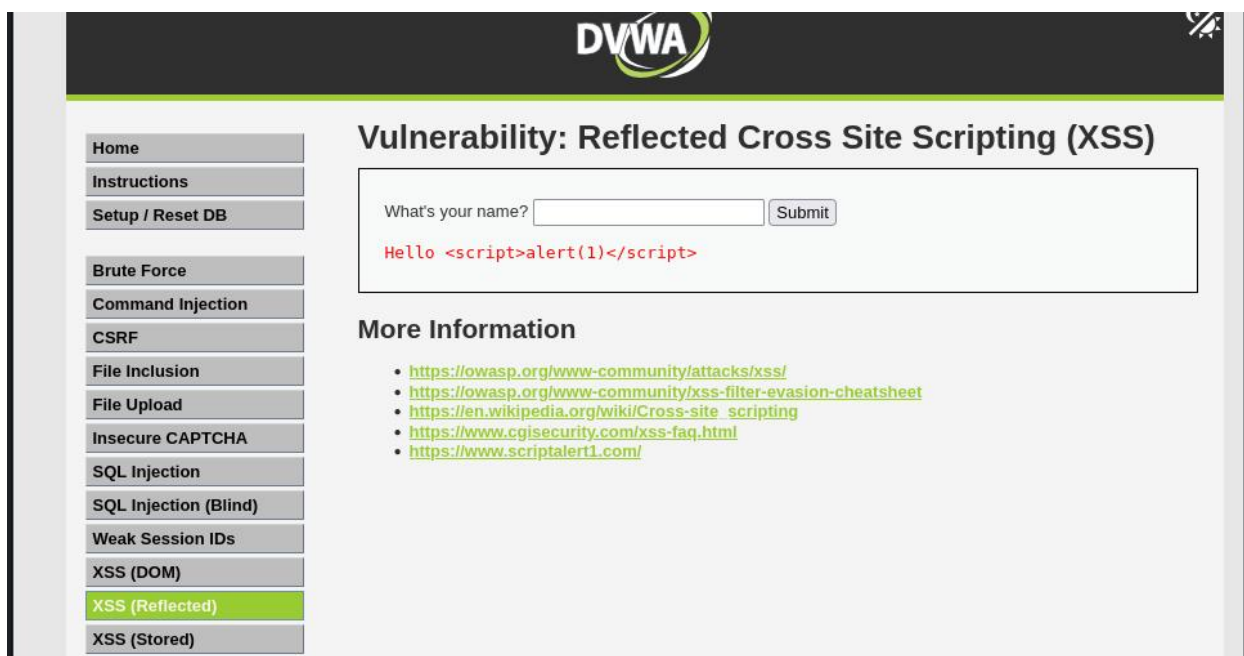
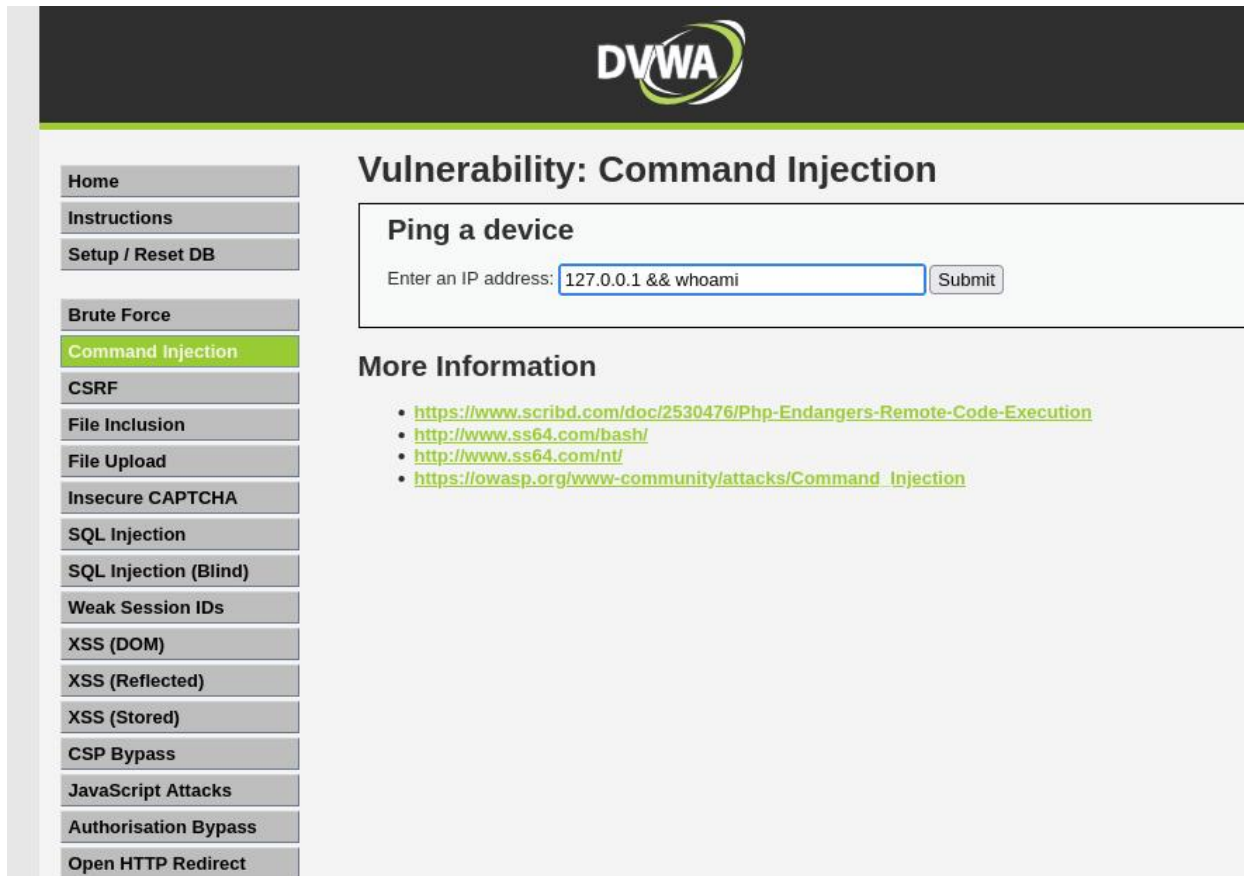


Figure 36 - The payload is reflected back into the response, confirming the vulnerability.

10.5 Command injection

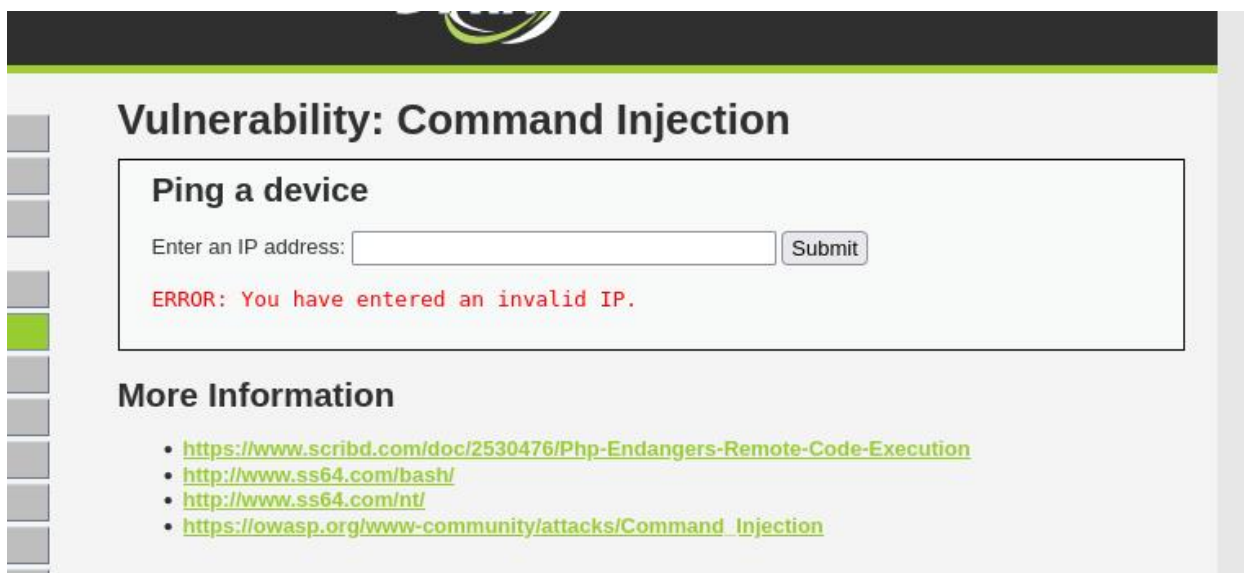
The command injection module passes user input to a system ping command. Appending a shell operator causes a second command to run.

Enter an IP address: 127.0.0.1 && whoami



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The left sidebar contains a menu with options: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection (highlighted), CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, and Open HTTP Redirect. The main content area is titled "Vulnerability: Command Injection" and features a "Ping a device" section. Below this title, there is a text input field labeled "Enter an IP address:" containing the payload "127.0.0.1 && whoami" and a "Submit" button. Below the input field, there is a "More Information" section with a list of links: <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>, <http://www.ss64.com/bash/>, <http://www.ss64.com/nt/>, and https://owasp.org/www-community/attacks/Command_injection.

Figure 37 - Command injection payload submitted.



The screenshot shows the DVWA interface with the "Vulnerability: Command Injection" section. The "Ping a device" section has an empty text input field labeled "Enter an IP address:" and a "Submit" button. Below the input field, there is a red error message: "ERROR: You have entered an invalid IP." Below the error message, there is a "More Information" section with a list of links: <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>, <http://www.ss64.com/bash/>, <http://www.ss64.com/nt/>, and https://owasp.org/www-community/attacks/Command_injection.

Figure 38 - A second variant rejected by input validation. Both attempts were made to see how each appeared in the log - Finding F13 records the outcome, and it is not the one a reader might expect.

10.6 Directory enumeration and user-agent spoofing

Reconnaissance was then simulated from the command line using curl, requesting paths that commonly exist on misconfigured servers and would be highly valuable to an attacker if present.

```
curl http://localhost/admin
curl http://localhost/.git
```

```
(kali@kali)-[/var/www/html/DVWA/config]
$ curl http://localhost/admin
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at localhost Port 80</address>
</body></html>

(kali@kali)-[/var/www/html/DVWA/config]
$
```

Figure 39 - Request for /admin returns 404 Not Found.

```
(kali@kali)-[/var/www/html/DVWA/config]
$ curl http://localhost/.git
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at localhost Port 80</address>
</body></html>
```

Figure 40 - Request for /.git returns 404. An exposed .git directory would leak the entire application source, which is why scanners always check it.

Finally, a request was sent with a deliberately falsified user agent to reproduce the identity-spoofing behaviour of automated tooling.

```
curl -A "sqlmap/1.7" http://localhost/DVWA
```

```
(kali@kali)-[/var/www/html/DVWA/config]
$ curl -A "sqlmap/1.7" http://localhost/DVWA
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="http://localhost/DVWA/">here</a>.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at localhost Port 80</address>
</body></html>

(kali@kali)-[/var/www/html/DVWA/config]
$
```

Figure 41 - Request carrying a spoofed sqlmap user agent, returning a 301 redirect.

10.7 The resulting log

```

127.0.0.1 - - [03/Aug/2026:07:02:17 -0400] "GET /DVWA/dvwa/images/theme-light-dark.png HTTP/1.1" 200 1315 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:17 -0400] "GET /DVWA/dvwa/images/spanner.png HTTP/1.1" 200 749 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:18 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
11 - - [03/Aug/2026:07:02:23 -0400] "OPTIONS HTTP/1.0" 200 126 "-" "Apache/2.4.68 (Debian) (internal dummy connection)"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "GET /DVWA/login.php HTTP/1.1" 200 945 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "GET /DVWA/dvwa/images/login_logo.png HTTP/1.1" 384 250 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "GET /DVWA/dvwa/css/login.css HTTP/1.1" 200 741 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:51 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:51 -0400] "GET /DVWA/setup.php HTTP/1.1" 200 2227 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:03:15 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:03:15 -0400] "GET /DVWA/login.php HTTP/1.1" 200 981 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/setup.php HTTP/1.1" 200 2328 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/dvwa/css/main.css HTTP/1.1" 200 1682 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/dvwa/js/add_event_listeners.js HTTP/1.1" 200 619 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/dvwa/js/dwafage.js HTTP/1.1" 200 900 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/dvwa/images/logo.png HTTP/1.1" 200 5338 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/dvwa/images/theme-light-dark.png HTTP/1.1" 200 1314 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/dvwa/images/spanner.png HTTP/1.1" 200 748 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:04 -0400] "POST /DVWA/setup.php HTTP/1.1" 302 343 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:04 -0400] "GET /DVWA/setup.php HTTP/1.1" 200 2521 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /DVWA/login.php HTTP/1.1" 200 953 "http://localhost/DVWA/setup.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /DVWA/dvwa/css/login.css HTTP/1.1" 200 741 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /DVWA/dvwa/images/login_logo.png HTTP/1.1" 384 250 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:12 -0400] "POST /DVWA/login.php HTTP/1.1" 302 481 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:12 -0400] "GET /DVWA/index.php HTTP/1.1" 200 2624 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:12 -0400] "GET /DVWA/dvwa/images/lock.png HTTP/1.1" 200 1045 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:43 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1707 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:22 -0400] "GET /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 1839 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:31 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2183 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:35 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:28 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2183 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

```

Figure 42 - The raw access log after the attack sequence. Unreadable at a glance - which is precisely why the structured analysis in Section 11 is necessary.

11. Part B Findings

Findings continue the numbering from Part A. Each was produced by a command run against the log the attacks generated, and each is checked against what was actually done.

F8 - Traffic profile and status distribution

Purpose: establish the baseline. In a 97-line log, the shape of the traffic is itself informative.

```
cd /var/log/apache2
wc -l access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr
awk '{print $9}' access.log | sort | uniq -c
```

```
Session Actions Edit View Help
(kali@kali)-[~]
$ ls
Desktop Documents Downloads Music Pictures Projects Public Templates Videos
(kali@kali)-[~]
$ cd /var/log/apache2
(kali@kali)-[/var/log/apache2]
$ ls
access.log error.log other_vhosts_access.log
```

Figure 43 - Locating the Apache log directory.

```
(kali@kali)-[~]
$ cd /var/log/apache2
(kali@kali)-[/var/log/apache2]
$ ls
access.log error.log other_vhosts_access.log
(kali@kali)-[/var/log/apache2]
$ wc -l access.log
97 access.log
(kali@kali)-[/var/log/apache2]
$ awk '{print $1}' access.log | sort | uniq -c | sort -nr
80 127.0.0.1
17 ::1
(kali@kali)-[/var/log/apache2]
$ awk '{print $9}' access.log | sort | uniq -c
68 200
3 301
8 302
2 304
16 404
```

Figure 44 - Record count, source distribution and status-code distribution.

Observation	Count	Meaning
Total records	97	Small enough to verify every conclusion by hand
From 127.0.0.1	80	IPv4 loopback - the browser-driven attacks
From ::1	17	IPv6 loopback - the curl-driven enumeration
Status 200	68	Successful responses - including every injection attack
Status 301	3	Redirects - includes the spoofed-user-agent request
Status 302	8	Login form redirects - the brute-force POSTs
Status 304	2	Cached assets
Status 404	16	Not found - the directory enumeration

Interpretation - and it is uncomfortable. Every one of the five attacks succeeded, yet 70% of the log is HTTP 200. The only attack that produced error responses was the one that found nothing. An analyst filtering for errors would investigate the harmless reconnaissance and miss the SQL injection, the XSS and the credential attack completely.

The single most transferable lesson from Part B

In Dataset A, hostile activity appeared as 404s because it was reconnaissance that failed. In Dataset B, hostile activity appeared as 200s because it was exploitation that worked. Error codes find attacks that miss; they are structurally incapable of finding attacks that land. Any detection strategy built only on status codes is calibrated to catch the attacks that do not matter.

Note also the split by address family. The browser used 127.0.0.1 and curl used ::1 - the same machine appearing as two different sources. Any per-IP threshold would count them separately and could easily miss a threshold that the combined activity would have crossed. Correlating IPv4 and IPv6 forms of the same host is a real operational requirement, not a lab artefact.

F9 - A real brute-force attack produced zero 401 responses

Hypothesis tested: Finding F1 predicted that application-level login attacks would be invisible to a status-code filter. Part B tests that directly, because a genuine brute-force attempt was performed in Section 10.1.

```
grep "login" access.log
```

```
(kali@kali)-[/var/log/apache2]
$ grep "login" access.log
127.0.0.1 - - [03/Aug/2026:06:48:10 -0400] "GET /DVWA/login.php HTTP/1.1" 200 956 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:06:48:11 -0400] "GET /DVWA/dvwa/css/login.css HTTP/1.1" 200 741 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:06:48:11 -0400] "GET /DVWA/dvwa/images/login_logo.png HTTP/1.1" 200 9375 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:06:48:11 -0400] "GET /favicon.ico HTTP/1.1" 404 527 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:17 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:17 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "GET /DVWA/login.php HTTP/1.1" 200 984 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "GET /DVWA/dvwa/images/login_logo.png HTTP/1.1" 200 9375 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "GET /DVWA/dvwa/css/login.css HTTP/1.1" 200 741 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:51 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:02:51 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:03:15 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:03:15 -0400] "GET /DVWA/login.php HTTP/1.1" 200 981 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "GET /DVWA/setup.php HTTP/1.1" 200 2328 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /DVWA/dvwa/css/login.css HTTP/1.1" 200 741 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /DVWA/dvwa/images/login_logo.png HTTP/1.1" 200 9375 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:09 -0400] "GET /favicon.ico HTTP/1.1" 404 527 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:12 -0400] "POST /DVWA/login.php HTTP/1.1" 302 481 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:07:12 -0400] "GET /DVWA/index.php HTTP/1.1" 200 2824 "http://localhost/DVWA/login.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

Figure 45 - Every request touching the login endpoint. The POST requests return 302, and no 401 appears anywhere.

Result: the status distribution in Figure 44 confirms it - the log contains 200, 301, 302, 304 and 404, and no 401 at all. The prediction from Part A holds under a real attack.

```
127.0.0.1 - - [03/Aug/2026:07:02:17 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482
127.0.0.1 - - [03/Aug/2026:07:02:41 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482
127.0.0.1 - - [03/Aug/2026:07:02:51 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482
127.0.0.1 - - [03/Aug/2026:07:03:15 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482
127.0.0.1 - - [03/Aug/2026:07:06:55 -0400] "POST /DVWA/login.php HTTP/1.1" 302 482
```

Interpretation: the application returns 302 on every login POST, successful or not, because it redirects in both cases. The response size is identical at 482 bytes. From the access log alone, these five events are indistinguishable from one another and from a legitimate user mistyping a password five times.

What can still be detected: the volume and the rhythm. Five POSTs to a login endpoint from one source inside four minutes is a behavioural signal even when every individual request looks normal. This is the basis of Rule R3 Condition B, and it is the only thing that works when the status code carries no information.

F10 - Successful authentication revealed by response size and follow-on request

The brute-force module tells a richer story than login.php. Filtering to that endpoint shows the full attack sequence:

```
grep "brute" access.log
```

```
(kali@kali)-[/var/log/apache2]
$ grep "brute" access.log
127.0.0.1 - - [03/Aug/2026:07:08:22 -0400] "GET /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 1839 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:31 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2103 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:08:35 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:20 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2103 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/js/dvwaPage.js HTTP/1.1" 200 900 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/css/main.css HTTP/1.1" 200 1682 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/js/add_event_listeners.js HTTP/1.1" 200 619 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/images/logo.png HTTP/1.1" 200 5330 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:24 -0400] "GET /DVWA/dvwa/images/theme-light-dark.png HTTP/1.1" 200 1314 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:34 -0400] "POST /DVWA/vulnerabilities/brute/ HTTP/1.1" 200 2103 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [03/Aug/2026:07:09:38 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

Figure 46 - The complete brute-force sequence against the DVWA module.

07:08:31	POST	/DVWA/vulnerabilities/brute/	200	2103	
07:09:20	POST	/DVWA/vulnerabilities/brute/	200	2103	
07:09:34	POST	/DVWA/vulnerabilities/brute/	200	2103	
07:09:44	POST	/DVWA/vulnerabilities/brute/	200	2145	<-- size changes
07:09:44	GET	/DVWA/hackable/users/admin.jpg	200	3829	<-- avatar loaded
07:10:11	POST	/DVWA/vulnerabilities/brute/	200	2105	
07:10:21	POST	/DVWA/vulnerabilities/brute/	200	2038	

The finding: three consecutive failures returned exactly 2,103 bytes. The fourth attempt returned 2,145 bytes - and was immediately followed by a request for /DVWA/hackable/users/admin.jpg, the account avatar image, which the application only renders once a user is authenticated.

Why this matters enormously: the status code is 200 for every single one of these requests. Success and failure are identical in field 9. But they are not identical in field 10, the response size, and they are not identical in what the browser fetches next. Two signals that most analysts ignore are, in this case, the only signals that exist.

Two generalisable detection techniques

First, response-size differential: when an application returns the same status code for success and failure, the byte count usually differs because the pages differ. A cluster of identical sizes followed by one outlier is a strong success indicator. Second, the post-authentication artefact: successful logins trigger resources that failed logins never request - an avatar, a dashboard widget, a session-scoped asset. Detecting the artefact is often easier and more reliable than detecting the login itself.

Honest caveat: response size is not a universal rule. Applications that pad responses, apply compression, or render dynamic content of varying length will produce noisy sizes. The technique is a strong lead, not a verdict, and should escalate an alert rather than raise one on its own.

The important secondary observation: look at what happens next. The browser then requested /DVWA/favicon.ico, and because the user was on the XSS page, the payload travelled along in the Referer header:

```
127.0.0.1 - - [03/Aug/2026:07:18:29 -0400] "GET /DVWA/favicon.ico HTTP/1.1" 200 1706  
"http://localhost/DVWA/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%281%29..."
```

Why this is a trap: a naive detector that searches the whole log line for script will flag this favicon request as an XSS attack. It is not an attack - it is an ordinary image request whose referrer happens to contain the payload from the previous page. Section 12.4 shows this false positive occurring in the analyst's own Python script, which is precisely why it is worth documenting.

Match on the field, not on the line

The fix is to restrict payload matching to the request path and query string - field 7 - rather than searching the entire record. A payload in the request is an attack; the same payload in the Referer header is an after-effect of an attack that has already been logged. Counting both doubles the alert volume for a single event and, worse, teaches the analyst that the rule is noisy.

F13 - Command injection left no usable trace (blind spot)

What was attempted: the payload 127.0.0.1 && whoami was submitted to the DVWA command injection module in Section 10.5, and it executed.

What the log shows: a single ordinary-looking POST.

```
127.0.0.1 - - [03/Aug/2026:07:19:07 -0400] "POST /DVWA/vulnerabilities/exec/
HTTP/1.1" 200 1883 "http://localhost/DVWA/vulnerabilities/exec/"
```

Why: the DVWA command injection form submits by POST, and Apache's access log records only the request line. The POST body - which is where the payload lives - is never written. There is no encoding to decode and no pattern to match, because the data simply is not there.

Interpretation: this is the most important negative finding in the report, and it is a genuine architectural limitation rather than an oversight. It also explains why SQL injection and XSS were detectable in this exercise while command injection was not: those two were submitted by GET, so their payloads landed in the query string. The same SQL injection sent by POST would have been equally invisible.

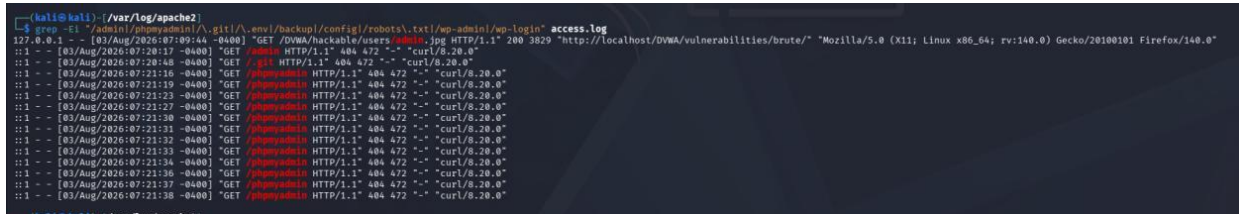
What would be required to detect it: one of the following. A web application firewall such as ModSecurity, which inspects request bodies before they reach the application. Application-level logging that records the parameters the code actually received. Or process-level telemetry on the server, since a shell spawned by the web-server user is an extremely strong compromise indicator regardless of how the request reached it.

State your blind spots explicitly

A findings report that lists only what was detected implies that everything else was absent. It was not. Recording that command injection succeeded and left no trace is more valuable to a reader than any of the successful detections, because it tells them where their current visibility ends and what they would need to buy or build to close it.

F14 - Directory enumeration from a second source address

```
grep -Ei "/admin|phpmyadmin|\\.git|\\.env|/backup|/config|/robots\\.txt|/wp-admin|/wp-login"
access.log
```



```
(kali@kali) /var/log/apache2
$ grep -Ei "/admin|phpmyadmin|\\.git|\\.env|/backup|/config|/robots\\.txt|/wp-admin|/wp-login" access.log
127.0.0.1 - - [03/Aug/2026:07:09:44 -0400] "GET /DVWA/hackable/users/whoami.jpg HTTP/1.1" 200 3829 "http://localhost/DVWA/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
::1 - - [03/Aug/2026:07:20:48 -0400] "GET /.git HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:16 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:19 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:23 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:27 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:30 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:31 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:32 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:33 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:34 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:36 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:37 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
::1 - - [03/Aug/2026:07:21:38 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"
```

Figure 50 - Sensitive-path requests. All originate from ::1, all return 404, and all carry the curl user agent.

Reformatting the same records with awk makes the pattern unmistakable:

```
grep -Ei "/admin|phpmyadmin|\\.git|\\.env|/backup|/config" access.log \
| awk -F'"' '{split($1,a," "); printf "\nIP: %s\nTime: %s\nRequest: %s\nStatus: %s\nUser-Agent: %s\n", \
a[1], substr($1,index($1,"[")+1,index($1,""])-index($1,"[")-1), $2, $3, $6}'
```

```
(kali@kali)-[/var/log/apache2]
$ grep -Ei "/admin|/phpmyadmin|/\.git|/\.env|/backup|/config" access.log \
| awk -F' ' '{split($1,a,""); printf "\nIP: %s\nTime: %s\nRequest: %s\nStatus: %s\nUser-Agent: %s\n",a[1],substr($1,index($1,"")+1,index($1,"")-index($1,"")-1),$2,$3,$5}'
IP: 127.0.0.1
Time: 03/Aug/2026:07:09:44 -0400
Request: GET /DVWA/hackable/users/admin.jpg HTTP/1.1
Status: 200 3829
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

IP: ::1
Time: 03/Aug/2026:07:20:17 -0400
Request: GET /admin HTTP/1.1
Status: 404 472
User-Agent: curl/8.20.0

IP: ::1
Time: 03/Aug/2026:07:20:48 -0400
Request: GET /.git HTTP/1.1
Status: 404 472
User-Agent: curl/8.20.0

IP: ::1
Time: 03/Aug/2026:07:21:16 -0400
Request: GET /phpmyadmin HTTP/1.1
Status: 404 472
User-Agent: curl/8.20.0

IP: ::1
Time: 03/Aug/2026:07:21:19 -0400
Request: GET /phpmyadmin HTTP/1.1
Status: 404 472
User-Agent: curl/8.20.0

IP: ::1
Time: 03/Aug/2026:07:21:23 -0400
Request: GET /phpmyadmin HTTP/1.1
Status: 404 472
User-Agent: curl/8.20.0

IP: ::1
Time: 03/Aug/2026:07:21:27 -0400
Request: GET /phpmyadmin HTTP/1.1
Status: 404 472
User-Agent: curl/8.20.0
```

Figure 51 - The same records rendered field by field. Presenting evidence readably is part of the analyst's job, not a cosmetic extra.

```
grep -Ei "/admin|/\.git|/\.env|/backup|/config|/phpmyadmin" access.log \
| awk '{print $1}' | sort | uniq
```

```
(kali@kali)-[~/SOC_Log_Analysis]
$ grep -Ei "/admin|/\.git|/\.env|/backup|/config|/phpmyadmin" access.log | awk '{print $1}' | sort | uniq
::1
127.0.0.1
```

Figure 52 - Two source addresses touched sensitive paths: ::1 (the curl scan) and 127.0.0.1 (the browser, incidentally).

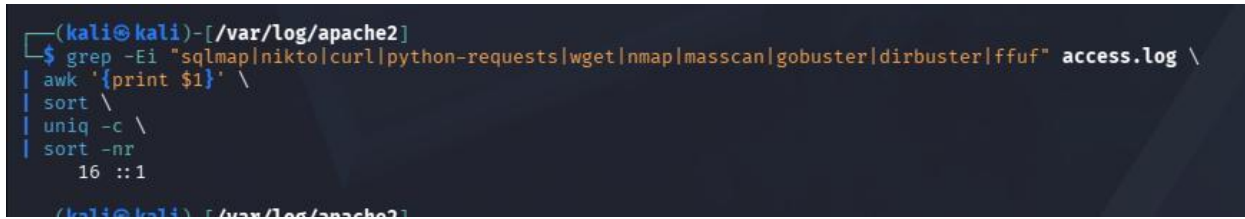
The behavioural fingerprint: four characteristics identify this as automated scanning, and every one of them is visible in Figure 51.

- **Identical response size on every miss.** 404 472 repeats exactly. The server is returning the same error page each time, which means every path was absent - the scanner is guessing, not following links.
- **No referrer.** Every request shows a dash. A browser following a link always sends one; a tool constructing URLs from a wordlist has nothing to send.
- **Machine-paced timing.** Requests at 07:21:16, 07:21:19, 07:21:23, 07:21:27, 07:21:30, 07:21:31, 07:21:32. Intervals of one to four seconds sustained without variation - a human clicking would produce far more irregular gaps.
- **Repetition of the same target.** /phpmyadmin was requested repeatedly. A person who received 404 once would not ask again eight times; a script iterating a list will.

Cross-reference to Part A: this is the same behaviour class as Finding F3, observed from the opposite side. In Part A the analyst inferred scanning from the shape of the data. Here the analyst knows it was scanning, and the same four characteristics are present - which confirms the Part A inference was sound.

F15 - Tool and spoofed user agents

```
grep -Ei "sqlmap|nikto|curl|python-requests|wget|nmap|masscan|gobuster|dirbuster|ffuf"
access.log \
| awk '{print $1}' | sort | uniq -c | sort -nr
```



```
(kali@kali)-[/var/log/apache2]
$ grep -Ei "sqlmap|nikto|curl|python-requests|wget|nmap|masscan|gobuster|dirbuster|ffuf" access.log \
| awk '{print $1}' \
| sort \
| uniq -c \
| sort -nr
16 ::1
```

Figure 53 - Sixteen requests from ::1 carry a tooling user agent.

User agent observed	How it arose	Detection value
curl/8.20.0	curl's honest default identity	Reliable when the tool does not bother to hide - which is most of the time
sqlmap/1.7	Deliberately spoofed with curl -A	Proves the field is attacker-controlled and can say anything
Mozilla/5.0 Firefox/140.0	... The analyst's real browser	The same person as the curl traffic, presenting a clean identity

Interpretation: all three identities in this log belong to one person on one machine. The browser attacks looked completely legitimate by user agent while succeeding at SQL injection and XSS; the curl scan looked obviously automated while finding nothing. The correlation between how suspicious a client looks and how much damage it does is, in this dataset, exactly zero.

Why the signal is still worth collecting: most real scanning traffic does not bother to spoof, so the check is cheap and catches a lot of low-effort activity. But it must never be trusted as a primary detection. Rule R5 in Section 15 rates it Low on its own and promotes it only when it correlates with another rule firing from the same source - which is exactly what happened here.

A deliberate demonstration

Sending curl -A "sqlmap/1.7" was not an attack. It was a test of the analyst's own detection: if the rule can be fooled into reporting sqlmap when no sqlmap exists, then it can equally be evaded by a real attacker who sets a Chrome string. Testing your rule's failure mode is part of writing it.

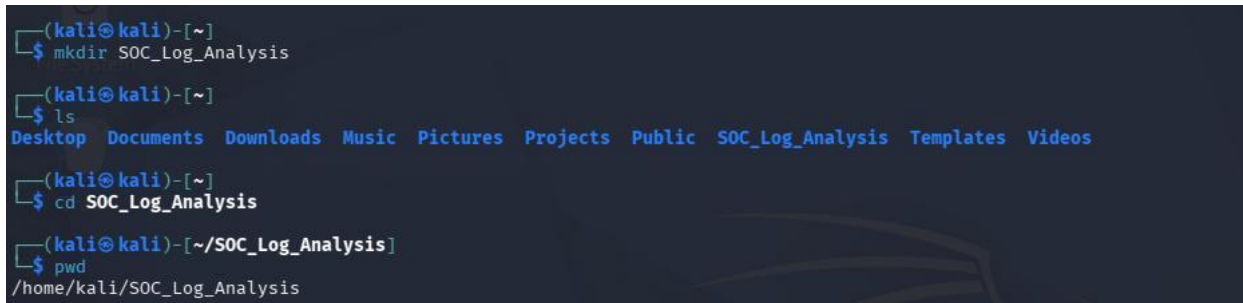
12. Building a Python Detection Engine

The shell one-liners in Section 11 proved each finding, but they share two limits: they cannot decode percent-encoded payloads without enumerating every encoded form by hand, and they leave the analyst assembling results manually. A small set of Python scripts fixes both. Full source for every script appears in Appendix B.

12.1 Preparing the analysis workspace

The log is copied out of `/var/log/apache2` into a working directory. Analysing a copy rather than the live file is standard practice: the original stays intact as evidence and cannot be corrupted by a mistake.

```
mkdir SOC_Log_Analysis
cd SOC_Log_Analysis
```



```
(kali㉿kali)-[~]
$ mkdir SOC_Log_Analysis

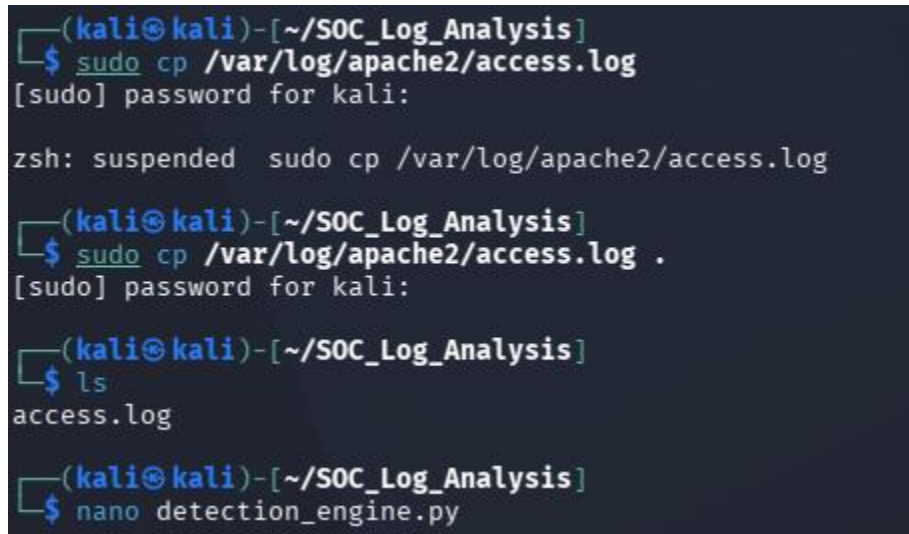
(kali㉿kali)-[~]
$ ls
Desktop  Documents  Downloads  Music  Pictures  Projects  Public  SOC_Log_Analysis  Templates  Videos

(kali㉿kali)-[~]
$ cd SOC_Log_Analysis

(kali㉿kali)-[~/SOC_Log_Analysis]
$ pwd
/home/kali/SOC_Log_Analysis
```

Figure 54 - Creating the dedicated analysis workspace.

```
sudo cp /var/log/apache2/access.log .
sudo chmod 644 access.log
nano detection_engine.py
```



```
(kali㉿kali)-[~/SOC_Log_Analysis]
$ sudo cp /var/log/apache2/access.log
[sudo] password for kali:

zsh: suspended sudo cp /var/log/apache2/access.log

(kali㉿kali)-[~/SOC_Log_Analysis]
$ sudo cp /var/log/apache2/access.log .
[sudo] password for kali:

(kali㉿kali)-[~/SOC_Log_Analysis]
$ ls
access.log

(kali㉿kali)-[~/SOC_Log_Analysis]
$ nano detection_engine.py
```

Figure 55 - Copying the log and beginning the first detection script. The `chmod` is needed because Apache logs are readable only by root and the `adm` group.

12.2 Brute-force detection

The first script counts requests to the DVWA brute-force endpoint per source address and alerts above a threshold.

```
from collections import defaultdict

LOG_FILE = "access.log"
```

```
login_attempts = defaultdict(int)

with open(LOG_FILE, "r") as file:
    for line in file:
        if "/DVWA/vulnerabilities/brute/" in line:
            ip = line.split()[0]
            login_attempts[ip] += 1

for ip, count in login_attempts.items():
    if count >= 5:
        print(f"[ALERT] Possible Brute Force Attack")
        print(f"Source IP : {ip}")
        print(f"Attempts : {count}")
```

```
(kali@kali)-[~/SOC_Log_Analysis]
$ nano detection_engine.py

(kali@kali)-[~/SOC_Log_Analysis]
$ sudo chmod +r access.log

(kali@kali)-[~/SOC_Log_Analysis]
$ sudo chmod 644 access.log

(kali@kali)-[~/SOC_Log_Analysis]
$ python3 detection_engine.py

=====
BRUTE FORCE DETECTION
=====

[ALERT] Possible Brute Force Attack
Source IP : 127.0.0.1
Attempts : 18
```

Figure 56 - The brute-force detector fires on 127.0.0.1 with 18 recorded hits.

Result: the attack is detected. This is the rule that Section 11 F9 showed no status-code filter could ever produce, so the script closes a real gap.

Honest limitation - the count of 18 is an overcount

The script counts every line containing the brute-force path, which includes the module's CSS, JavaScript and image files loaded alongside the form. The actual number of login submissions was around seven POSTs. The alert is correct, but the number attached to it is not, and an analyst who quoted 18 login attempts in an incident report would be wrong. Filtering on POST requests only - if \$6 is POST - fixes it, and Rule R3 in Section 15 specifies that filter.

12.3 Suspicious user-agent detection

```
bad_agents = ["sqlmap", "nikto", "curl", "python-requests",
              "wget", "ffuf", "gobuster", "dirbuster"]

with open("access.log") as file:
    for line in file:
        lower = line.lower()
        for agent in bad_agents:
            if agent in lower:
                print("[ALERT] Suspicious User-Agent")
                print(line)
```



```

kali@kali:~/SOC_Log_Analysis$ nano detection_suspicious_user.py
kali@kali:~/SOC_Log_Analysis$ python3 detection_suspicious_user.py

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:20:17 -0400] "GET /admin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:20:48 -0400] "GET /.git HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:16 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:19 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:23 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:27 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:30 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:31 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

[ALERT] Suspicious User-Agent
::1 - - [03/Aug/2026:07:21:32 -0400] "GET /phpmyadmin HTTP/1.1" 404 472 "-" "curl/8.20.0"

```

Figure 57 - Every curl-driven request is flagged, with the offending line printed for context.

Assessment: it works, and printing the full matching line is good practice - an alert without evidence forces the analyst to go back to the log anyway. Two weaknesses are worth naming. First, curl is a legitimate tool used by monitoring systems and API clients, so this list will produce false positives in any real environment. Second, matching against the whole lowercased line rather than the user-agent field means a URL containing the word curl would also match.

12.4 Cross-site scripting detection

This script decodes each line before matching, which is what makes it succeed where a plain grep would fail.

```

import urllib.parse

patterns = ["<script", "alert(", "onerror", "onload",
           "<svg", "<img", "javascript:"]

with open("access.log", "r", errors="ignore") as file:
    for line in file:
        decoded = urllib.parse.unquote_plus(line.lower())
        if any(pattern in decoded for pattern in patterns):
            print("[HIGH] XSS Attack Detected")
            print(decoded)

```

```

(kali@kali)-[~/SOC_Log_Analysis]
$ python3 detection_XSS.py

=====
[HIGH] XSS Attack Detected
=====
127.0.0.1 - - [03/aug/2026:07:18:29 -0400] "get /dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>&user_token=17a4fb0f4f132b9b51088d7e7a40a005 http/1.1" 200 1911 "http://localhost/dvwa/vulnerabilities/xss_r/" "mozilla/5.0 (x11; linux x86_64; rv:140.0) gecko/20100101 firefox/140.0"

=====
[HIGH] XSS Attack Detected
=====
127.0.0.1 - - [03/aug/2026:07:18:29 -0400] "get /dvwa/favicon.ico http/1.1" 200 1706 "http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>&user_token=17a4fb0f4f132b9b51088d7e7a40a005" "mozilla/5.0 (x11; linux x86_64; rv:140.0) gecko/20100101 firefox/140.0"

=====
[HIGH] XSS Attack Detected
=====
127.0.0.1 - - [03/aug/2026:07:18:47 -0400] "get /dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>&user_token=aa60c3c1706e6f97da53ca9f17023783 http/1.1" 200 1913 "http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>&user_token=17a4fb0f4f132b9b51088d7e7a40a005" "mozilla/5.0 (x11; linux x86_64; rv:140.0) gecko/20100101 firefox/140.0"

=====
[HIGH] XSS Attack Detected
=====
127.0.0.1 - - [03/aug/2026:07:18:47 -0400] "get /dvwa/favicon.ico http/1.1" 200 1706 "http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>&user_token=aa60c3c1706e6f97da53ca9f17023783" "mozilla/5.0 (x11; linux x86_64; rv:140.0) gecko/20100101 firefox/140.0"

=====
[HIGH] XSS Attack Detected
=====
127.0.0.1 - - [03/aug/2026:07:19:07 -0400] "get /dvwa/vulnerabilities/exec/ http/1.1" 200 1883 "http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>&user_token=aa60c3c1706e6f97da53ca9f17023783" "mozilla/5.0 (x11; linux x86_64; rv:140.0) gecko/20100101 firefox/140.0"

```

Figure 58 - XSS detections. Note the second and fourth alerts: both are requests for favicon.ico.

The decoding step is the whole point. `unquote_plus` turns `%3Cscript%3E` back into `<script>`, so the pattern list can be written in readable form instead of enumerating every possible encoding. This is the single clearest advantage Python holds over grep for this task.

The false positives are equally instructive. Two of the five alerts are `favicon.ico` requests. As Finding F12 explained, those are ordinary image loads whose `Referer` header carries the payload from the previous page. The script searches the entire line, so it matches the referrer as readily as the request. The fix is to decode and match only the request path - field 7 - which would reduce these five alerts to three genuine ones.

Why the false positive is reported rather than quietly fixed

The script was run as written, the output is shown as produced, and the flaw is explained. A report that silently corrected the script would hide the most useful lesson in this section: that a detection which looks like it is working can still be inflating its own numbers by 40%, and only reading the alerts individually reveals it.

12.5 Directory enumeration detection

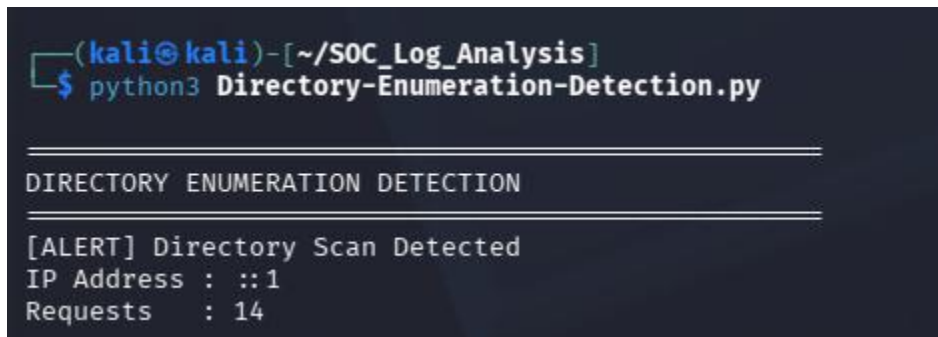
```
from collections import defaultdict

scan_paths = ["/admin", "/.git", "/.env", "/backup",
              "/config", "/phpmyadmin"]

scan_count = defaultdict(int)

with open("access.log") as file:
    for line in file:
        ip = line.split()[0]
        for path in scan_paths:
            if path in line:
                scan_count[ip] += 1

for ip, count in scan_count.items():
    if count >= 5:
        print("[ALERT] Directory Scan Detected")
        print(f"IP Address : {ip}")
        print(f"Requests   : {count}")
```



```
(kali@kali)-[~/SOC_Log_Analysis]
$ python3 Directory-Enumeration-Detection.py

=====
DIRECTORY ENUMERATION DETECTION
=====
[ALERT] Directory Scan Detected
IP Address : ::1
Requests   : 14
```

Figure 59 - Directory scanning detected from ::1 with 14 matching requests.

Assessment: correct detection, and the source matches the ground truth from Section 10.6 exactly. It carries the same weakness that Part A Finding F7 identified in the shell version: it counts requests rather than distinct paths, so eight requests for /phpmyadmin count as eight. In this log that inflates the number but does not change the verdict. In a production log, a single broken link polled by a monitoring system would trigger it daily.

12.6 SQL injection detection

```
import urllib.parse

with open("access.log", "r") as file:
    for line in file:
        if "/sql/" in line:
            decoded = urllib.parse.unquote_plus(line)
            print(decoded)
```

Assessment: this one decodes and prints, but it filters on the DVWA-specific path /sql/ rather than on injection patterns. It works perfectly in this lab and would find nothing at all on any other application. The version in the prevention engine improves on it by matching decoded content against a pattern list - union, select, information_schema, sleep(), benchmark(), drop and so on - which is portable to any application. Both are included in Appendix B so the progression is visible.

12.7 Honest assessment of the detection scripts

Every script produced the correct verdict. None of them produced a fully correct count. That distinction matters, because an alert that fires correctly with an inflated number still misleads the person who has to act on it.

Script	Weakness as written	Correction applied in Section 15
detection_engine.py	Counts assets as login attempts (18 vs ~7)	R3 filters on POST method only
detection_XSS.py	Matches the Referer header, inflating alerts by 40%	R4 matches the request field only
detection_suspicious_user.py	Matches the whole line, not the UA field; curl is legitimate	R5 matches the UA field and is Low severity alone
Directory-Enumeration-Detection.py	Counts requests, not distinct paths	R1 counts distinct paths
detection_SQL.py	Filters on a DVWA-specific path	R4 matches decoded injection patterns

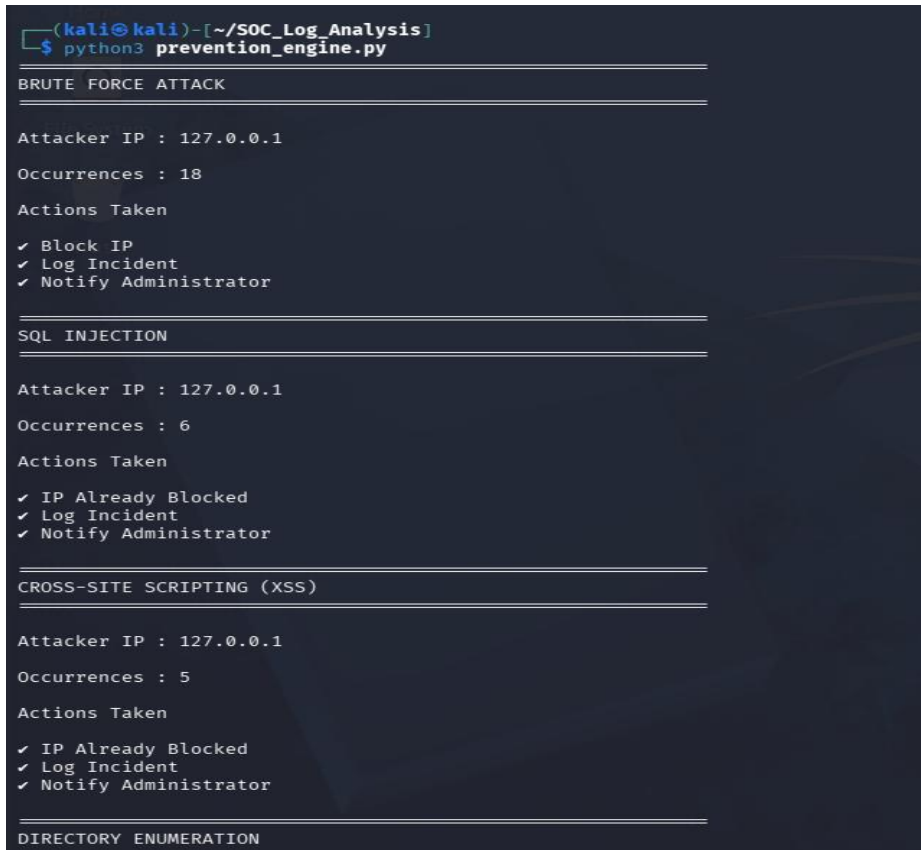
13. Automated Response - prevention_engine.py

The final script consolidates all five detections into one pass over the log and adds a response stage: it maintains a set of offending addresses, records each incident to a file, and reports what action was taken. Full source appears in Appendix B.

13.1 Design

- A single pass reads each line once, lowercases it, decodes it, and evaluates all five detection categories - far more efficient than five separate scripts each re-reading the file.
- A shared blocked_ips set means an address already actioned by an earlier detection is reported as already blocked rather than blocked twice.
- Every action is appended to security_incidents.log with a timestamp, so the run leaves an auditable record.
- A closing summary counts distinct attackers per category, which is the number an incident responder actually needs.

13.2 Output



```
(kali㉿kali)~[~/SOC_Log_Analysis]
$ python3 prevention_engine.py

=====
BRUTE FORCE ATTACK
=====

Attacker IP : 127.0.0.1
Occurrences : 18
Actions Taken
✓ Block IP
✓ Log Incident
✓ Notify Administrator

=====
SQL INJECTION
=====

Attacker IP : 127.0.0.1
Occurrences : 6
Actions Taken
✓ IP Already Blocked
✓ Log Incident
✓ Notify Administrator

=====
CROSS-SITE SCRIPTING (XSS)
=====

Attacker IP : 127.0.0.1
Occurrences : 5
Actions Taken
✓ IP Already Blocked
✓ Log Incident
✓ Notify Administrator

=====
DIRECTORY ENUMERATION
=====
```

Figure 60 - Brute force, SQL injection and XSS detected, with the action taken recorded for each.

```

=====
DIRECTORY ENUMERATION
=====

Attacker IP : ::1
Occurrences : 14
Actions Taken
✓ Block IP
✓ Log Incident
✓ Notify Administrator

=====
SUSPICIOUS USER-AGENT
=====

Attacker IP : ::1
Occurrences : 16
Actions Taken
✓ IP Already Blocked
✓ Log Incident
✓ Notify Administrator

=====
PREVENTION SUMMARY
=====

Total Brute Force Attackers      : 1
Total SQL Injection Attackers    : 1
Total XSS Attackers              : 1
Total Directory Scan Attackers   : 2
Total Suspicious User-Agent Attackers : 1

Blocked IP Addresses
✓ ::1
✓ 127.0.0.1

=====
PREVENTION COMPLETED SUCCESSFULLY
=====

```

Figure 61 - Directory enumeration and suspicious user agents, followed by the consolidated prevention summary.

Attack category	Source	Count	Action recorded
Brute Force Attack	127.0.0.1	18	Block IP, log incident, notify administrator
SQL Injection	127.0.0.1	6	IP already blocked, log incident, notify
Cross-Site Scripting	127.0.0.1	5	IP already blocked, log incident, notify
Directory Enumeration	::1	14	Block IP, log incident, notify
Suspicious User-Agent	::1	16	IP already blocked, log incident, notify

All five attack classes were detected from a 97-line log. Two addresses were flagged - which is correct, since the attacks came from a browser on 127.0.0.1 and curl on ::1.

13.3 The incident record

```

(kali@kali)-[~/SOC_Log_Analysis]
$ cat security_incidents.log
2026-08-03 13:59:36.144834 | SQL Injection | 127.0.0.1
2026-08-03 13:59:36.147363 | Suspicious User-Agent | ::1
2026-08-03 14:13:16.403504 | Brute Force Attack | 127.0.0.1 | Occurrences : 18
2026-08-03 14:13:16.403693 | SQL Injection | 127.0.0.1 | Occurrences : 6
2026-08-03 14:13:16.403731 | Cross-Site Scripting (XSS) | 127.0.0.1 | Occurrences : 5
2026-08-03 14:13:16.403767 | Directory Enumeration | ::1 | Occurrences : 14
2026-08-03 14:13:16.403801 | Suspicious User-Agent | ::1 | Occurrences : 16

```

Figure 62 - security_incidents.log, written automatically by the engine. Two runs are visible, which is itself a useful observation about idempotency.

A detail worth noting: the file contains entries from two separate executions, appended rather than replaced. That is the correct behaviour for an audit log, but it means the file cannot be used as a live list of currently-blocked addresses without deduplication. Distinguishing an append-only audit trail from a current-state list is a design decision worth making explicitly rather than by accident.

13.4 Where the word prevention is doing too much work

The engine detects and records. It does not block.

The script prints 'Block IP' and adds the address to a Python set that exists only for the lifetime of the process. No firewall rule is created, no traffic is stopped, and when the script exits the set is gone. This is a detection and reporting engine with a response stage modelled, which is a perfectly reasonable thing to build - but naming it prevention risks giving a reader the impression that something is being enforced.

Turning it into genuine prevention would require, at minimum:

1. Writing blocks to something durable - an iptables or nftables rule, an ipset, or a fail2ban jail - so they survive the process.
2. An expiry mechanism, since permanent blocks from an automated rule eventually lock out a legitimate user behind a shared address.
3. An allow-list checked before any block, so monitoring systems, internal scanners and known administrator addresses can never be blocked.
4. A dry-run mode, so the rule can be observed for a week before it is permitted to act. Every one of the counting errors documented in Section 12.7 would have caused a wrongful block on day one.

That last point is the important one. The detection logic was right in every case and the counts were wrong in four out of five. Automated blocking amplifies a counting error into an outage, which is why detection is deployed in monitoring mode long before it is given permission to act.

SYNTHESIS

14. Consolidated Findings Summary

ID	Finding	Dataset	Evidence	Severity	Rule
F1	No HTTP 401 responses - status-code blind spot for app-level logins	A	Figure 7	Info	R3
F2	Elevated 404 rate - 213 of 10,000 (2.13%)	A	Figures 8-9	Low	R1
F3	WordPress / Joomla / Drupal admin enumeration across rotated prefixes	A	Figures 10-11	Medium	R2
F4	HEAD-method probing for legacy FCKeditor file manager	A	Figure 11	Medium	R2, R5
F5	File-inclusion and traversal probe strings in request paths	A	Figure 11	Medium	R2, R4
F6	Implausible obsolete UA; automation agent; unverified crawler claim	A	Figures 8, 10	Low	R5
F7	Failures concentrated in two sources - one benign, one suspicious	A	Figure 12	Low	R1
F8	70% of attack traffic returned HTTP 200 - errors find only failed attacks	B	Figure 44	High	R4, R6
F9	Real brute force produced zero 401s - F1 prediction confirmed	B	Figures 44-45	High	R3
F10	Successful login revealed by response size and avatar request	B	Figure 46	High	R3, R6
F11	SQL injection succeeded; payload present only percent-encoded	B	Figure 47	High	R4
F12	XSS succeeded; payload contaminates Referer and inflates alerts	B	Figures 48-49	High	R4
F13	Command injection succeeded and left no trace in the access log	B	Section 11	High	None - gap
F14	Directory enumeration -	B	Figures 50-52	Medium	R1, R2

	404s, no referrer, machine-paced timing				
F15	Tool user agent and deliberately spoofed identity	B	Figure 53	Low	R5

Severity basis: Dataset A findings are rated no higher than Medium because every probe returned 404 - nothing succeeded. Dataset B findings are rated High because the corresponding attacks did succeed, and because each represents a class of activity that a status-code-based detection strategy would miss entirely. F13 is rated High with no rule attached, because the gap is real and honesty about it is more useful than a rule that would not work.

15. Recommended Detection Rules

The task asks for two to three rules. Six are supplied, because the two datasets together exposed six distinct behaviours, and because three of them cover attacks that the other three structurally cannot see. Each rule states its logic, its threshold, the reasoning behind that threshold, and its expected false positives - a rule delivered without tuning guidance will be switched off by whoever has to answer its alerts.

R1 - Distinct-path 404 enumeration from a single source

Detects	Directory and content discovery scanning (F2, F3, F7, F14)
Logic	One client IP receives 404s for 10 or more DISTINCT paths within 5 minutes
Severity	Medium
Data needed	Client IP, status code, request path, timestamp

```
awk '$9==404 {print $1, $7}' access.log \
| sort -u \
| awk '{print $1}' \
| uniq -c \
| awk '$1 >= 10 {print $1" distinct 404 paths from \"$2\"}' \
| sort -nr
```

The design decision that makes this rule work: it counts distinct paths, not total requests. Finding F6 showed the noisiest source in Dataset A - 60 failures - requested one single path repeatedly. Under a total-request threshold it alerts constantly and the rule becomes worthless; under a distinct-path threshold its count is 1 and it never fires, while a scanner walking a wordlist crosses the threshold immediately. Finding F14 confirms the same weakness in the Python version, where eight requests for /phpmyadmin counted as eight.

Threshold rationale: ten distinct missing paths in five minutes is far outside normal behaviour - a person following stale links might hit two or three across a whole session. Start at ten, measure a week, tighten to five if the environment is quiet.

Expected false positives: search-engine crawlers after a site restructure; broken CDN or asset pipelines; your own security team's scanners. Mitigate by allow-listing verified crawler ranges and internal scanner sources, not by raising the threshold.

R2 - Sensitive administrative path probing

Detects	Targeted admin and login discovery, and known-vulnerable component probing (F3, F4, F5, F14)
Logic	One client IP requests 3 or more DISTINCT sensitive paths within 10 minutes, at ANY status
Severity	High if any such request returns 200 or 302; otherwise Medium
Data needed	Client IP, request path, status code, timestamp

```
grep -E -i '(wp-login|wp-admin|/administrator|/admin\.php|/admin/|phpmyadmin|fckeditor|/user/register|\.env|\.git|/config|\.|/setup\.php|/backup)' access.log \
| awk '{print $1, $7}' \
| sort -u \
| awk '{print $1}' | uniq -c \
| awk '$1 >= 3 {print "ALERT: \"$2\" probed \"$1\" sensitive paths"}'
```

Why status code is deliberately ignored: R1 only sees failures. If an attacker guesses correctly and the admin panel actually exists, the response is 200 and R1 goes silent at exactly the moment it matters most. R2 fires on the attempt regardless of outcome and escalates when a probe succeeds. The two rules are complementary by design, not redundant.

Proven against both datasets: 195.250.34.144 in Dataset A requested /wp-login.php, /administrator/ and /admin.php within twenty-six seconds - exactly three distinct sensitive paths, and note that its total 404 count was below ten, so R1 alone would have missed it. In Dataset B the curl scan touched /admin, /.git and /phpmyadmin, also crossing the threshold.

Expected false positives: administrators logging in from a new address, and monitoring tools polling a login page. Mitigate by allow-listing known administrative sources and by requiring distinct paths rather than alerting on any single hit.

R3 - Login endpoint pressure and credential attack

Detects	Brute force and credential stuffing - the gap proven by F1 and F9
Logic	Condition A: 10+ responses with status 401 or 403 from one IP in 5 minutes. Condition B: 5+ POST requests to a login endpoint from one IP in 5 minutes, at any status. Condition C: 3+ POSTs to one endpoint returning an identical response size, followed by one of a different size.
Severity	High. Critical if Condition C fires, which indicates the attack succeeded.
Data needed	Client IP, method, path, status, response size, timestamp

```
# Condition A - HTTP-layer authentication failures
awk '$9 == 401 || $9 == 403 {print $1}' access.log \
| sort | uniq -c \
| awk '$1 >= 10 {print "AUTH-FAIL BURST: \"$2\" - \"$1\" failures\"}'

# Condition B - application-layer login pressure (POST only, per F9)
awk '$6 ~ /POST/ && $7 ~ /(login|signin|wp-login|auth|brute)/ {print $1}' access.log \
| sort | uniq -c \
| awk '$1 >= 5 {print "LOGIN PRESSURE: \"$2\" - \"$1\" POSTs\"}'

# Condition C - response-size change indicating success (per F10)
awk '$6 ~ /POST/ && $7 ~ /(login|brute|auth)/ {print $1, $7, $10}' access.log \
| sort | uniq -c | sort -k2,3
```

Why three conditions rather than one: each covers what the others cannot. Condition A handles servers that use HTTP authentication. Condition B handles everything else - and F9 proved that everything else is the normal case, since a real brute force against DVWA produced zero 401s. Condition C addresses the hardest problem of all: F10 showed that success and failure returned the identical status code, and were separable only by response size.

The POST filter is not optional. Section 12.2 counted 18 brute-force events where roughly 7 login submissions occurred, because it counted the module's stylesheets and images too. Restricting to POST is the difference between an accurate incident report and an inflated one.

Expected false positives: users who have forgotten a password, and shared-NAT addresses such as a corporate gateway where many people appear as one IP. Mitigate by keying on username as well as IP where the application log provides it, and by using a rolling window rather than a fixed one.

R4 - Injection payload in the request, after decoding

Detects	SQL injection and cross-site scripting attempts (F5, F8, F11, F12)
Logic	URL-decode the request path and query string, then match against SQL and script pattern lists. Match the REQUEST FIELD ONLY, never the whole line.
Severity	High. Critical if the response status is 200 and the response size differs from the endpoint's baseline.
Data needed	Client IP, request path with query string, status, response size

```
import urllib.parse

SQL = ["' or ", " union ", " select ", "information_schema",
       "sleep(", "benchmark(", " drop ", " insert ", " delete "]
XSS = ["<script", "alert(", "onerror", "onload", "<svg", "<img", "javascript:"]

with open("access.log", errors="ignore") as f:
    for line in f:
        parts = line.split(' ')
        if len(parts) < 2:
            continue
        request = urllib.parse.unquote_plus(parts[1].lower()) # field 7 only
        ip = line.split()[0]
        if any(p in request for p in SQL):
            print(f"[HIGH] SQL injection attempt from {ip}: {request}")
        if any(p in request for p in XSS):
            print(f"[HIGH] XSS attempt from {ip}: {request}")
```

The two corrections built into this rule: first, it decodes before matching, because F11 showed the payload exists in the log only as %27+OR+%271%27%3D%271 and a plain-text search finds nothing. Second, it splits the line and matches only the request field, because F12 showed that searching the whole line also matches the Referer header and inflated the analyst's own XSS alerts by 40%.

Why the severity escalates on 200: F8 established that successful exploitation returns 200 while failed reconnaissance returns 404. An injection payload that produced a 200 with an unusual response size is the strongest single indicator in this entire report that something worked.

Expected false positives: search boxes where users legitimately type words like select or union; documentation sites hosting code samples; security researchers testing your own forms. Mitigate by excluding known search endpoints and by requiring metacharacters - quotes, angle brackets, comment markers - rather than keywords alone.

R5 - Anomalous or tooling user agent

Detects	Scanning tools and forged clients (F6, F15)
Logic	The user-agent FIELD is empty or '-', OR matches a known tool string, OR claims a browser more than five years older than the log date
Severity	Low alone. Promote to Medium when it correlates with R1, R2 or R4 firing from the same source.
Data needed	Client IP, user-agent field, timestamp

```
awk -F' ' '{print $1, $6}' access.log \
| grep -E -i '(sqlmap|nikto|nmap|masscan|dirbuster|gobuster|wpscan|ffuf|
acunetix|curl|wget|python-requests|libwww-perl|MSIE [1-6]\.|Windows NT 5\.)' \
| awk '{print $1}' | sort | uniq -c | sort -nr
```


Why this rule is rated Low on its own: the user-agent string is set by the client and can say anything. F15 demonstrated this deliberately - a single curl command reported itself as sqlmap/1.7, and no sqlmap was involved. A competent attacker copies a current Chrome string and this rule never fires. Meanwhile the genuinely damaging attacks in Dataset B all arrived from a real Firefox browser with a completely clean user agent.

Why it is still worth collecting: most real scanning traffic does not bother to spoof, so the check is nearly free and catches a large volume of low-effort activity. Its proper role is to raise confidence in an alert another rule already produced, not to raise alerts by itself.

Note the field restriction: extracting field 6 after splitting on double quotes matches the user agent specifically. The analyst's own script in Section 12.3 matched the whole lowercased line, which would also fire on a URL containing the word curl.

R6 - Post-exploitation success indicator

Detects	The transition from attempted to successful - the case F8 and F10 showed all other rules miss
Logic	A source that triggered R1, R2, R3 or R4 subsequently receives a 200 or 302 on a sensitive endpoint, OR requests a post-authentication resource it had not previously requested
Severity	Critical - this is an incident, not an alert
Data needed	Client IP, path, status, response size, timestamp, plus alert history

```
# All activity from a flagged source, in order - the timeline an analyst needs
SUSPECT="127.0.0.1"
grep "^$SUSPECT " access.log \
| awk '{print $4, $6, $7, $9, $10}' \
| sed 's/\[//'
```

Why this rule exists: R1 through R5 all detect attempts. None of them answers the only question that actually matters during an incident, which is whether anything worked. F10 provides the template: three failures at 2,103 bytes, one response at 2,145, then a request for an avatar image that only loads when authenticated. The success was invisible in the status code and obvious in the sequence.

Generalising the pattern: identify, for each application, the resources that only load after authentication or after privileged access - a dashboard asset, a user avatar, an admin-only stylesheet, a session-scoped endpoint. A request for one of those from a source that was brute-forcing sixty seconds earlier is not an alert to triage. It is a confirmed compromise.

Expected false positives: low by construction, since the rule requires a prior alert from the same source. The main risk is shared addresses, where a legitimate user behind the same NAT gateway succeeds while somebody else was attacking. Correlate on session or username where available.

16. Limitations and Analyst Notes

Stating the boundaries of an analysis is part of the analysis. The following constraints apply to the conclusions in this report.

- **Access logs only.** No error log, firewall log, application log or process telemetry was available. Finding F13 shows exactly what that costs: a successful command injection left no trace at all.
- **No response-body visibility.** A 200 tells you the request succeeded, not what was returned. Data exfiltration through a working SQL injection would look like ordinary traffic in this dataset.
- **No POST body visibility.** Any attack submitted by POST is invisible to payload matching. The SQL injection and XSS in Part B were detectable only because DVWA submits those particular forms by GET.
- **Dataset B is small and self-generated.** 97 records from one analyst on one machine. It proves each detection can fire and that the counts are inflated; it says nothing about false-positive rates at production volume.
- **Dataset A is a sampled window.** 10,000 records over a limited period. A slow scan deliberately paced under any per-window threshold would not appear as a cluster.
- **Loopback addresses are not realistic sources.** 127.0.0.1 and ::1 would be allow-listed in any real deployment. The detection logic transfers; the specific addresses do not.
- **No attribution.** Source IPs are recorded as observed. They may be proxies, VPN exits, compromised hosts or shared-NAT addresses. The address is where the packet came from, not who sent it.
- **Thresholds are unvalidated starting points.** Every value in Section 15 is reasoned from these two datasets, not tuned against a production baseline. They should run in monitoring mode for at least a week before being allowed to page anyone, and far longer before being allowed to block.

17. Conclusion and Next Steps

The exercise achieved its objective and then went past it. A raw 10,000-line web-server log was reduced to seven evidenced findings using nothing beyond the standard Unix toolchain. A second dataset was then generated deliberately so that each finding could be tested against known ground truth, producing eight more findings and six detection rules with defined thresholds and documented tuning guidance.

Dataset A showed automated reconnaissance: enumeration of WordPress, Joomla and Drupal administrative endpoints across rotated path prefixes, HEAD-method probing for a legacy file manager with known upload vulnerabilities, file-inclusion test strings, and at least one client presenting an implausibly obsolete browser identity. Every probe returned 404 and no compromise indicator was found.

Dataset B showed the opposite: five attacks, all successful, of which four returned HTTP 200 and one left no trace at all.

The three lessons worth carrying forward

- **Error codes find only the attacks that failed.** Dataset A's hostile traffic appeared as 404s because it was reconnaissance that missed. Dataset B's appeared as 200s because it was exploitation that landed. A detection strategy built on error codes is calibrated to catch the attacks that do not matter.
- **Volume is a poor discriminator; composition is a good one.** The loudest source in Dataset A - 60 failures - was a broken automation client, while genuinely suspicious behaviour sat below any volume

threshold. Counting distinct paths instead of total requests separated the two cleanly, and that single design decision is the difference between a rule that gets deployed and a rule that gets muted.

- **The detection was right and the numbers were wrong.** All five Python detections produced the correct verdict; four produced an inflated count. Because a wrong count in a detection engine becomes a wrongful block in a prevention engine, monitoring mode is not a formality - it is what stands between a counting bug and an outage.

Next steps

5. Correct the five scripts per Section 12.7 - POST filter, request-field-only matching, distinct-path counting - and re-run to confirm the counts drop to the true values.
6. Extend the lab to cover POST-based injection and confirm the F13 blind spot, then install ModSecurity and measure how much of it closes.
7. Translate the six rules into Sigma format so they are portable to any SIEM rather than tied to these scripts.
8. Run all six rules against a larger public dataset in monitoring mode to obtain a realistic false-positive rate before proposing any of them for deployment.
9. Add IPv4 and IPv6 correlation, since Finding F8 showed one machine appearing as two independent sources.
10. Group-leader deliverable: consolidate the group's Week 2 submissions into a single status update and send it to the Team Lead by Friday.

Appendix A - Full Command Reference

Every command used, grouped by phase, so a reader can reproduce the work from a clean system.

A.1 Part A - setup, acquisition and validation

```
cd Downloads && mkdir log-analysis && cd log-analysis
wget
https://raw.githubusercontent.com/elastic/examples/master/Common%20Data%20Formats/apache_logs/ap
ache_logs
ls -lh                # confirm size on disk
head apache_logs      # confirm Combined Log Format
wc -l apache_logs     # establish the record-count baseline
mv apache_logs access.log
```

A.2 Part A - analysis

```
# F1 - authentication failures
grep ' 401 ' access.log | wc -l

# F2 - not-found volume
grep ' 404 ' access.log | wc -l

# F3, F4, F5 - which paths failed, ranked by frequency
grep ' 404 ' access.log | awk -F'"' '{print $2}' | sort | uniq -c | sort -nr

# F7 - sources exceeding the failure threshold
awk '$9==404 {print $1}' access.log | sort | uniq -c | awk '$1>10'

# Automation
nano detect_404.sh && chmod +x detect_404.sh && ./detect_404.sh
```

A.3 Part B - lab build

```
sudo apt update && sudo apt upgrade -y
sudo apt install apache2 -y
sudo systemctl enable apache2 && sudo systemctl start apache2
systemctl status apache2
sudo apt install php php-mysqli php-gd php-curl php-xml php-mbstring \
    libapache2-mod-php unzip git -y
sudo systemctl enable mariadb
sudo mysql # CREATE DATABASE dvwa; CREATE USER ...; GRANT ...; FLUSH PRIVILEGES;
cd /var/www/html && sudo rm index.html
sudo git clone https://github.com/digininja/DVWA.git
sudo chown -R www-data:www-data /var/www/html/DVWA
sudo chmod -R 755 /var/www/html/DVWA
cd /var/www/html/DVWA/config
sudo cp config.inc.php.dist config.inc.php && sudo nano config.inc.php
sudo systemctl restart apache2
```

A.4 Part B - attack simulation

```
# Through the browser at http://localhost/DVWA (security level: Low)
# Brute Force      -> repeated wrong passwords, then correct one
# SQL Injection    -> 1' OR '1'='1
# XSS (Reflected) -> <script>alert(1)</script>
# Command Injection -> 127.0.0.1 && whoami

# From the command line
curl http://localhost/admin
curl http://localhost/.git
curl -A "sqlmap/1.7" http://localhost/DVWA
```

A.5 Part B - log analysis

```
cd /var/log/apache2
wc -l access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr      # F8 sources
awk '{print $9}' access.log | sort | uniq -c                # F8 status codes
grep "brute" access.log                                     # F10
grep "login" access.log                                     # F9
grep -Ei "union|or%20|or\+|select|information_schema" access.log # F11
grep -Ei "script|alert|onerror|onload|svg|img|iframe|javascript:" access.log # F12
grep -Ei "%3C|%3E|%22|%27|%3D" access.log                  # F12 encoded forms
grep -Ei "/admin|phpmyadmin|/.git|/.env|/backup|/config" access.log # F14
grep -Ei "sqlmap|nikto|curl|python-requests|wget|nmap|gobuster" access.log \
| awk '{print $1}' | sort | uniq -c | sort -nr             # F15
```

A.6 Part B - Python engine

```
mkdir ~/SOC_Log_Analysis && cd ~/SOC_Log_Analysis
sudo cp /var/log/apache2/access.log .
sudo chmod 644 access.log
python3 detection_engine.py
python3 detection_SQL.py
python3 detection_XSS.py
python3 detection_suspicious_user.py
python3 Directory-Enumeration-Detection.py
python3 prevention_engine.py
cat security_incidents.log
```

A.7 Useful extensions not shown in the screenshots

```
# Full status distribution - establishes what normal looks like
awk '{print $9}' access.log | sort | uniq -c | sort -nr

# Top talkers by total request volume
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -20

# User-agent frequency - anomalies surface at the bottom of the list
awk -F'"' '{print $6}' access.log | sort | uniq -c | sort -nr | head -20

# Requests per hour - reveals bursts and off-hours activity
awk '{print $4}' access.log | cut -d: -f1-2 | uniq -c

# Everything one specific source did, in order
grep '^208.91.156.11 ' access.log | awk '{print $4, $6, $7, $9}'

# Non-GET methods - HEAD, PUT and DELETE deserve a second look
awk -F'"' '{print $2}' access.log | awk '{print $1}' | sort | uniq -c | sort -nr
```

Appendix B - Python Source Code

Source listings for the five detection scripts and the consolidated prevention engine, captured directly from the analysis machine.

B.1 detection_engine.py

```
(kali@kali)-[~/SOC_Log_Analysis]
$ cat detection_engine.py
from collections import defaultdict
import re

LOG_FILE = "access.log"

login_attempts = defaultdict(int)

with open(LOG_FILE, "r") as file:
    for line in file:
        if "/DVWA/vulnerabilities/brute/" in line:

            ip = line.split()[0]
            login_attempts[ip] += 1

print("="*50)
print("BRUTE FORCE DETECTION")
print("="*50)

for ip, count in login_attempts.items():
    if count >= 5:
        print(f"[ALERT] Possible Brute Force Attack")
        print(f"Source IP : {ip}")
        print(f"Attempts : {count}\n")
```

Figure 63 - Brute-force detector. Counts requests to the DVWA brute-force endpoint per source and alerts at five or more.

B.2 detection_SQL.py

```
(kali@kali)-[~/SOC_Log_Analysis]
$ cat detection_SQL.py
import urllib.parse

with open("access.log", "r") as file:
    for line in file:
        if "/sql/" in line:
            decoded = urllib.parse.unquote_plus(line)
            print(decoded)
```

Figure 64 - SQL injection extractor. Decodes matching lines with `unquote_plus` so the payload becomes readable.

B.3 detection_suspicious_user.py

```
(kali@kali)-[~/SOC_Log_Analysis]
$ cat detection_suspicious_user.py
bad_agents = [
    "sqlmap",
    "nikto",
    "curl",
    "python-requests",
    "wget",
    "ffuf",
    "gobuster",
    "dirbuster",
]

with open("access.log") as file:
    for line in file:
        lower = line.lower()
        for agent in bad_agents:
            if agent in lower:
                print("="*50)
                print("[ALERT] Suspicious User-Agent")
                print("="*50)
                print(line)
```

Figure 65 - User-agent detector, with the tool string list it matches against.

B.4 detection_XSS.py

```
(kali㉿kali)-[~/SOC_Log_Analysis]
$ cat detection_XSS.py
import urllib.parse

patterns = [
    "<script",
    "alert(",
    "onerror",
    "onload",
    "<svg",
    "<img",
    "javascript:"
]

with open("access.log", "r", errors="ignore") as file:
    for line in file:
        decoded = urllib.parse.unquote_plus(line.lower())
        if any(pattern in decoded for pattern in patterns):
            print("=" * 60)
            print("[HIGH] XSS Attack Detected")
            print("=" * 60)
            print(decoded)
```

Figure 66 - XSS detector. The decode-then-match order is what allows readable patterns to find encoded payloads.

B.5 Directory-Enumeration-Detection.py

```
(kali㉿kali)-[~/SOC_Log_Analysis]
$ cat Directory-Enumeration-Detection.py
from collections import defaultdict

LOG_FILE = "access.log"

scan_paths = [
    "/admin",
    "/.git",
    "/.env",
    "/backup",
    "/config",
    "/phpmyadmin"
]

scan_count = defaultdict(int)

with open(LOG_FILE) as file:
    for line in file:
        ip = line.split()[0]
        for path in scan_paths:
            if path in line:
                scan_count[ip] += 1

print("="*50)
print("DIRECTORY ENUMERATION DETECTION")
print("="*50)

for ip, count in scan_count.items():
    if count ≥ 5:
        print(f"[ALERT] Directory Scan Detected")
        print(f"IP Address : {ip}")
        print(f"Requests : {count}\n")
```

Figure 67 - Directory enumeration detector, counting sensitive-path requests per source address.

B.6 prevention_engine.py

The consolidated engine: one pass over the log, five detection categories, a shared blocked-address set and an audit trail.

```
(kali@kali)-[~/SOC_Log_Analysis]
$ cat prevention_engine.py
from collections import defaultdict
import urllib.parse
from datetime import datetime

LOG_FILE = "access.log"

# -----
# Dictionaries to store attack counts
# -----

bruteforce = defaultdict(int)
sql_attack = defaultdict(int)
xss_attack = defaultdict(int)
directory_scan = defaultdict(int)
user_agent_attack = defaultdict(int)

blocked_ips = set()

# -----
# Directory Enumeration Paths
# -----

scan_paths = [
    "/admin",
    "/.git",
    "/.env",
    "/backup",
    "/config",
    "/phpmyadmin"
]

# -----
# SQL Injection Patterns
# -----

sql_patterns = [
    "' or '",
    "' or '",
    " union ",
    " select ",
    "information_schema",
    "sleep(",
    "benchmark(",
    "drop ",
    "insert ",
    "delete "
```

Figure 68 - Imports, per-category counters, the shared `blocked_ips` set, scan paths and SQL patterns.

```

# -----
# XSS Patterns
# -----

xss_patterns = [
    "<script",
    "alert(",
    "onerror",
    "onload",
    "<svg",
    "<img",
    "javascript:"
]

# -----
# Suspicious User Agents
# -----

bad_agents = [
    "sqlmap",
    "nikto",
    "curl",
    "python-requests",
    "wget",
    "ffuf",
    "gobuster",
    "dirbuster"
]

# -----
# Read Apache Log
# -----

with open(LOG_FILE, "r", errors="ignore") as file:
    for line in file:
        ip = line.split()[0]
        lower = line.lower()
        decoded = urllib.parse.unquote_plus(lower)

        # -----
        # Brute Force Detection
        # -----

        if "/dvwa/vulnerabilities/brute/" in lower:

```

Figure 69 - XSS patterns and the tool user-agent list, followed by the single-pass read loop.

```

# -----
if "/dvwa/vulnerabilities/brute/" in lower:
    bruteforce[ip] += 1
# -----
# SQL Injection Detection
# -----

if "/sql/" in lower:
    if any(pattern in decoded for pattern in sql_patterns):
        sql_attack[ip] += 1

# -----
# XSS Detection
# -----

if any(pattern in decoded for pattern in xss_patterns):
    xss_attack[ip] += 1

# -----
# Directory Enumeration Detection
# -----

for path in scan_paths:
    if path in lower:
        directory_scan[ip] += 1

# -----
# Suspicious User-Agent Detection
# -----

for agent in bad_agents:
    if agent in lower:
        user_agent_attack[ip] += 1
        break

# -----
# Helper Function

```

Figure 70 - The five detection blocks. Each line is lowercased once and decoded once, then evaluated against every category.

```

# Helper Function
# -----

def take_action(attack_name, ip, count):

    print("=" * 60)
    print(attack_name.upper())
    print("=" * 60)

    print()
    print("Attacker IP :", ip)
    print()
    print("Occurrences :", count)
    print()
    print("Actions Taken")
    print()

    if ip not in blocked_ips:

        print("✓ Block IP")
        blocked_ips.add(ip)

    else:

        print("✓ IP Already Blocked")

    print("✓ Log Incident")
    print("✓ Notify Administrator")
    print()

    with open("security_incidents.log", "a") as log:

        log.write(
            f"{datetime.now()} | {attack_name} | {ip} | Occurrences : {count}\n"
        )

    with open("admin_alerts.log", "a") as admin:

        admin.write(
            f"{datetime.now()} | {attack_name} | {ip}\n"
        )

# -----
# BRUTE FORCE PREVENTION
# -----

for ip, count in bruteforce.items():

    if count ≥ 5:

```

Figure 71 - The take_action helper: prints the finding, adds the address to blocked_ips if new, and appends to both log files.

```
    if count ≥ 5:
        take_action(
            "Brute Force Attack",
            ip,
            count
        )

# -----
# SQL INJECTION PREVENTION
# -----
for ip, count in sql_attack.items():
    take_action(
        "SQL Injection",
        ip,
        count
    )

# -----
# XSS PREVENTION
# -----
for ip, count in xss_attack.items():
    take_action(
        "Cross-Site Scripting (XSS)",
        ip,
        count
    )

# -----
# DIRECTORY ENUMERATION PREVENTION
# -----
for ip, count in directory_scan.items():
    if count ≥ 5:
        take_action(
            "Directory Enumeration",
            ip,
            count
        )
```

Figure 72 - Per-category response loops for SQL injection, XSS and directory enumeration.


```

# -----
# SUSPICIOUS USER-AGENT PREVENTION
# -----

for ip, count in user_agent_attack.items():
    take_action(
        "Suspicious User-Agent",
        ip,
        count
    )

# -----
# FINAL SUMMARY
# -----

print("=" * 60)
print("PREVENTION SUMMARY")
print("=" * 60)
print()

print("Total Brute Force Attackers      :", len(bruteforce))
print("Total SQL Injection Attackers    :", len(sql_attack))
print("Total XSS Attackers                :", len(xss_attack))
print("Total Directory Scan Attackers     :", len(directory_scan))
print("Total Suspicious User-Agent Attackers :", len(user_agent_attack))
print()

print("Blocked IP Addresses")

if blocked_ips:
    for ip in blocked_ips:
        print("✓", ip)
else:
    print("No IP addresses were blocked.")

print()
print("=" * 60)
print("PREVENTION COMPLETED SUCCESSFULLY")
print("=" * 60)

```

Figure 73 - User-agent response loop and the closing summary, which counts distinct attackers per category.

Appendix C - Screenshot Index

All 73 figures are the analyst's own captures, taken on 02 and 03 August 2026 on the Kali Linux analysis machine.

Figures	Content	Section
1 - 6	Workspace setup, download, validation and rename of Dataset A	Section 4
7	401 search returning zero matches	Finding F1
8 - 9	Raw 404 records and the count of 213	Finding F2
10 - 11	Ranked failed request paths, parts 1 and 2	Findings F3, F4, F5
12	Source IPs exceeding the 404 threshold	Finding F7
13 - 14	detect_404.sh in nano, and its execution	Section 7
15 - 22	System update, Apache install and start, PHP stack, database creation	Sections 9.1 - 9.4
23 - 27	Web root preparation, DVWA clone, permissions, configuration, restart	Sections 9.5 - 9.6
28 - 30	DVWA login page, welcome page and security level	Sections 9.7 - 9.8
31 - 33	Brute force failure and success, first look at the log	Sections 10.1 - 10.2
34 - 38	SQL injection, reflected XSS input and output, command injection	Sections 10.3 - 10.5
39 - 42	curl directory probes, spoofed user agent, raw resulting log	Sections 10.6 - 10.7
43 - 44	Log location, record count, source and status distribution	Finding F8
45 - 46	Login endpoint traffic and the full brute-force sequence	Findings F9, F10
47 - 49	SQL injection and XSS payloads located, including encoded forms	Findings F11, F12
50 - 52	Sensitive-path requests, field-by-field rendering, source addresses	Finding F14
53	Tool user agents by source	Finding F15
54 - 55	Analysis workspace creation and log copy	Section 12.1
56 - 59	Detection engine outputs: brute force, user agent, XSS, directory scan	Sections 12.2 - 12.5
60 - 62	Prevention engine output and the incident record	Section 13
63 - 73	Python source listings for all six scripts	Appendix B